Universidad Siglo 21

Seminario de Práctica de Informática (INF275 - 46312)

Seguimiento de vulnerabilidades por proyecto

Trabajo práctico 4

Versión final del proyecto integrador

Daniel Sebastián Rodríguez

Legajo VINF016869 | DNI 43.731.653

Licenciatura en Informática | Modalidad EDH

Profesor Pablo Alejandro Virgolini

9 de noviembre de 2026

Índice

1. Título, introducción y justificación . . . 3
2. Definiciones del proyecto y del sistema . . . 3
3. Elicitación . . . 4
4. Conocimiento del negocio . . . 5
5. Propuesta de solución . . . 7
6. Requerimientos . . . 8
7. Casos de uso . . . 10
8. Etapa de análisis . . . 12
9. Etapa de diseño . . . 13
10. Etapa de implementación . . . 14
11. Etapa de pruebas . . . 15
12. Prototipos de interfaz . . . 17
13. Base de datos . . . 18
14. Definiciones de comunicación . . . 21
15. Desarrollo del prototipo en Java . . . 21
16. Versión final del sistema . . . 28
17. Trazabilidad y referencias . . . 37
Anexo A. Preguntas de elicitación . . . 37

Enlaces de la entrega

Repositorio público (código, base de datos y salidas en la carpeta TP4)	https://github.com/rodriguezd0/semprac-seguimiento-vulnerabilidades
Video de presentación (3 minutos)	https://drive.google.com/file/d/1WOmHzJpT8UlJvwH-wRaepPd_MIHo2izk/view?usp=sharing

1. Título, introducción y justificación

El proyecto se llama «Sistema de seguimiento de vulnerabilidades por proyecto». Propongo desarrollar una aplicación de escritorio en Java para registrar las vulnerabilidades que se detectan en los recursos informáticos, calcular su severidad con CVSS 3.1 y seguir su corrección hasta el cierre. Los hallazgos se organizan por proyecto y por activo (el administrador gestiona los registros y las cuentas, y cada cliente consulta solamente las vulnerabilidades de sus proyectos).

El contexto es el área tecnológica de Derechos Del Fútbol, donde hago tareas de administración de sistemas, bases de datos y herramientas internas. Elegí este tema porque tiene relación directa con mi trabajo y porque junta en una misma aplicación registro de datos, cálculo, permisos por rol y seguimiento de tareas.

1.1 Problema y justificación

Hoy los hallazgos de seguridad del área se anotan en una planilla y se comunican por correo o mensajería. Con pocos casos ese método alcanza, pero cuando hay varios proyectos abiertos la información de cada hallazgo (el recurso afectado, su gravedad, quién lo corrige y qué se hizo) queda repartida entre la planilla y los mensajes. La gravedad depende del criterio de quien anota y no queda constancia de cuándo cambió el estado ni por qué, así que cuando un referente pregunta por su proyecto la respuesta se arma a mano revisando las dos fuentes.

A eso se suma un problema de acceso. La planilla es un único archivo y compartirla con un referente implica mostrarle también los hallazgos de otros proyectos (un servidor mal configurado de un proyecto ajeno es justo el tipo de dato que no debería circular fuera del área). La aplicación resuelve las dos cuestiones con un mismo registro, porque cada hallazgo guarda sus datos, una severidad calculada con un método estándar y cada cambio de estado con su comentario, y sobre ese registro el acceso por proyecto deja que el cliente consulte lo suyo sin ver lo ajeno. En los términos de la lectura 1, las entradas son los hallazgos con sus proyectos y activos, el proceso es su registro, evaluación y seguimiento, y las salidas son las consultas y los listados (Saroka, 2002, como se citó en *Sistemas de información*, s. f.).

Este documento integra las cuatro entregas. Retoma la primera con las correcciones de la devolución (justificación, objetivos, elicitación, procesos de negocio, modelo de dominio y casos de uso), las etapas de análisis, diseño, implementación y pruebas, la base de datos, las definiciones de comunicación y el prototipo de consola del TP3 (sección 15), y cierra con la versión final en Swing conectada a MySQL (sección 16), donde también se resumen las correcciones de esta entrega.

2. Definiciones del proyecto y del sistema

En esta versión separo el objetivo del proyecto, lo que tiene que hacer el sistema y los fines de la organización a los que ambos aportan, tres cosas que antes aparecían mezcladas.

2.1 Objetivo general del proyecto

Centralizar el seguimiento de las vulnerabilidades de los recursos del área tecnológica de Derechos Del Fútbol para que, antes de diciembre de 2026, cada hallazgo tenga una severidad calculada con un criterio uniforme, un responsable y un historial verificable, y cada referente conozca el estado de sus proyectos sin acceder a información ajena.

2.2 Objetivos específicos

1. Contar con un registro único de hallazgos por proyecto y activo que reemplace a la planilla y a los mensajes.
2. Obtener la severidad de cada hallazgo con CVSS 3.1 base, con resultados iguales a los ejemplos oficiales de FIRST.
3. Conservar la trazabilidad de cada corrección (responsable, fecha objetivo y cada cambio de estado con su motivo).
4. Asegurar que cada cliente consulte solamente los hallazgos de los proyectos que tiene asignados.

2.3 Objetivo del sistema y fines de la organización

El sistema tiene que registrar los hallazgos de seguridad de cada proyecto, calcular su severidad CVSS 3.1, controlar su ciclo de estados (PENDIENTE, EN_TRATAMIENTO y CERRADA, con reapertura) y ofrecer a cada usuario las consultas que corresponden a su rol.

Derechos Del Fútbol trabaja en el ámbito jurídico deportivo y su fin es prestar ese servicio con la información de sus clientes protegida y disponible. El proyecto aporta a ese fin porque acorta el tiempo en que un recurso queda expuesto y deja constancia de cómo se trató cada problema (algo que también sirve ante una auditoría o un reclamo).

2.4 Alcance y límites

Aspecto	Definición
Límites	Desde el alta de un proyecto con su cliente y sus activos hasta el cierre verificado (o la reapertura) de cada hallazgo y la consulta de su historial.
Incluye	Cuentas con rol administrador o cliente, proyectos, activos y responsables, registro de hallazgos, cálculo CVSS 3.1 base, asignación con fecha objetivo, ciclo de estados con historial, consultas por proyecto, estado y severidad, exportación del listado a TXT y carga inicial de hallazgos desde la planilla del área (CSV).
No contempla	Escáneres o detección automática, explotación o remediación automática, métricas temporales y ambientales de CVSS, avisos por correo, acceso web o móvil e integración con otros sistemas.
Restricciones	Java y Swing, base MySQL y aplicación de escritorio en la red interna del área.

2.5 Aplicación del PUD

Aplico el PUD de forma iterativa e incremental, guiada por los casos de uso y centrada en la arquitectura MVC. Cada iteración retoma los artefactos anteriores y los corrige cuando hace falta (por eso este documento integra la definición del proyecto con los modelos nuevos). En el inicio se definieron el problema, los objetivos y el alcance. La elaboración cubre el negocio, los requerimientos, los casos de uso, el análisis, el diseño, el despliegue, las pruebas y la base de datos. La construcción tuvo dos incrementos, el prototipo de consola de la sección 15 (el modelo del diseño programado en Java y probado con los datos en memoria) y la versión final de la sección 16, con Swing y MySQL. La transición es la carga inicial de la planilla del área y la demostración con el usuario de referencia.

3. Elicitación

La elicitación consiste en reunir el conocimiento necesario para producir un modelo de los requerimientos (Loucopoulos, 1995, como se citó en *Sistemas de información*, s. f.). Como formo parte del área combiné la observación del trabajo diario con la revisión de los registros existentes, una entrevista con el responsable del área tecnológica y un cuestionario breve a los referentes de proyecto. Las preguntas y la síntesis de las respuestas están en el Anexo A.

3.1 Fuentes de información

Fuente	Técnica	Aporte al proyecto
Planilla de hallazgos y correos del área	Revisión documental	Qué datos se registran hoy y cuáles faltan (responsable, fecha objetivo e historial).
Responsable del área tecnológica	Entrevista (Anexo A.1)	Reglas de asignación, cierre y reapertura, y qué puede ver cada persona.
Referentes de proyecto	Cuestionario (Anexo A.2)	Qué necesitan consultar y con qué frecuencia.
Trabajo propio en el área	Observación participante	Recorrido real de un hallazgo desde la detección hasta la verificación.
Especificación CVSS 3.1 (FIRST, 2019a)	Revisión documental	Métricas base, fórmula, redondeo y categorías de severidad.

3.2 Actores e interesados

Los actores son el administrador (personal del área tecnológica que registra, evalúa, asigna y sigue los hallazgos, y que además da de alta las cuentas) y el cliente (referente de un proyecto, que solo consulta). El responsable de corrección es la persona o el equipo que corrige sobre el activo (soporte interno, desarrollo o un proveedor) y no usa el sistema (es un dato del catálogo). La dirección también es interesada porque le importa el nivel de exposición de la organización, aunque no opera el sistema y recibe el listado exportado.

3.3 Necesidades y requerimientos iniciales

Necesidad detectada	Requerimiento inicial	RF / RNF
Tener en un solo lugar todos los datos de un hallazgo.	Registrar hallazgos por proyecto y activo con sus datos mínimos.	RF01, RF03, RF09
Priorizar con un criterio común.	Calcular la severidad con CVSS 3.1 base.	RF12
Saber quién corrige y para cuándo.	Asignar responsable y fecha objetivo.	RF02, RF04
Reconstruir qué pasó con cada hallazgo.	Guardar cada cambio de estado con fecha, usuario y comentario.	RF06, RF07, RNF03
Que el referente vea su avance sin pedirlo.	Consultas filtradas con acceso por proyecto.	RF05, RNF04
Que un referente no vea hallazgos ajenos.	Cuentas con rol y control de acceso en cada consulta.	RF10, RF11, RNF04, RNF05
Entregar un resumen a la dirección.	Exportar el listado filtrado.	RF08

4. Conocimiento del negocio

Derechos Del Fútbol trabaja en el ámbito jurídico deportivo. El área tecnológica administra sus recursos informáticos (el sitio web, la aplicación interna de expedientes, las bases de datos, la red de la oficina y los respaldos) y el proceso que el proyecto informatiza es el seguimiento de los hallazgos de seguridad sobre esos recursos.

4.1 Proceso de negocio actual

El proceso relevado es el seguimiento de un hallazgo tal como se hace hoy, con tres roles (administrador del área, responsable de corrección y referente del proyecto). La figura 1 lo muestra como diagrama de actividades de UML con un carril por rol.

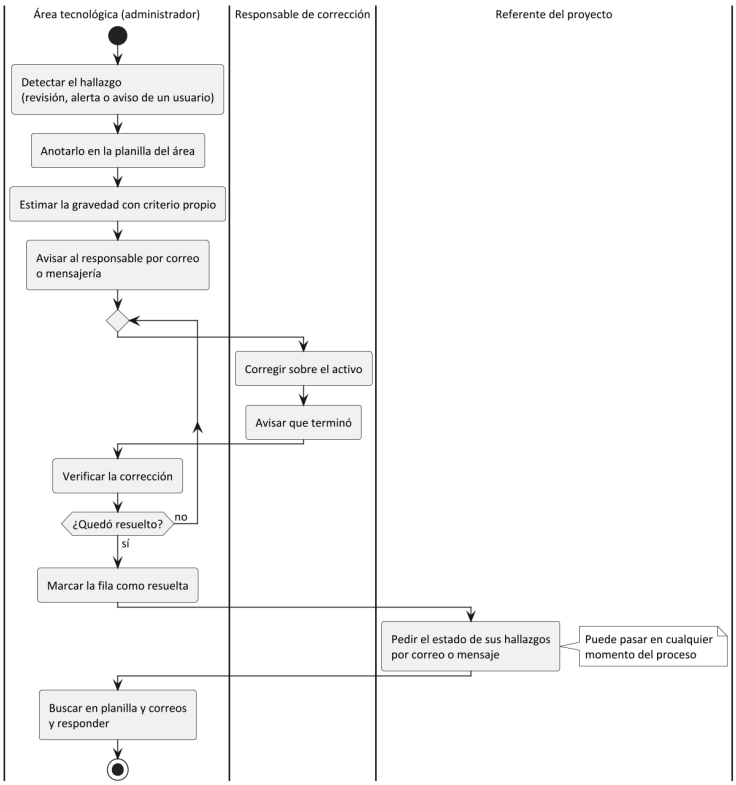


Figura 1. Proceso actual de seguimiento de un hallazgo. Elaboración propia.

El hallazgo aparece por una revisión propia, una alerta de un equipo o el aviso de un usuario. El administrador lo anota en la planilla con su propio criterio de gravedad y avisa al responsable, y el ciclo de corrección y verificación se repite hasta que el problema queda resuelto. En paralelo el referente pregunta cómo va su proyecto y la respuesta sale de revisar planilla y correos. De ese recorrido salen los problemas que el sistema tiene que resolver. Hallazgos parecidos reciben gravedades distintas porque no hay un método común, no queda registro de cuándo cambió el estado ni por qué, un hallazgo cerrado que reaparece se anota como nuevo (y se pierde su historia) y compartir la planilla expone hallazgos de otros proyectos.

4.2 Modelo de dominio

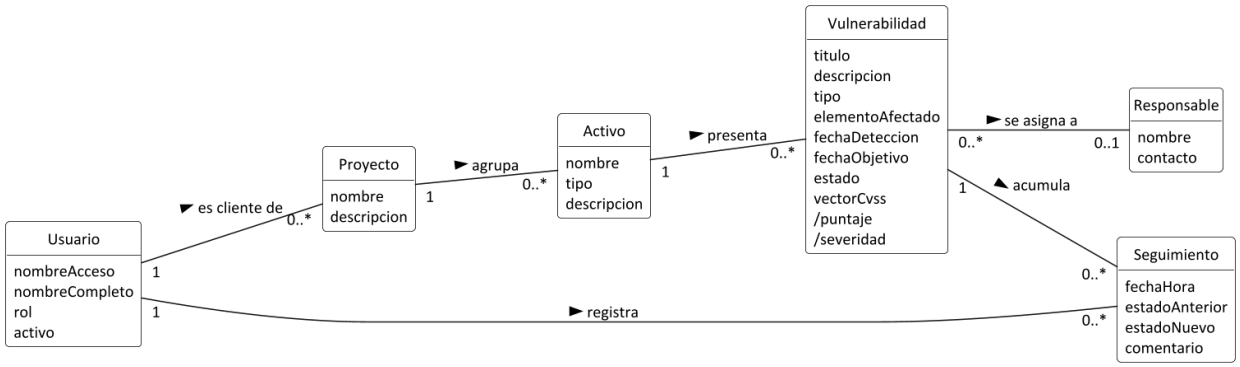


Figura 2. Modelo de dominio con clases conceptuales, atributos y multiplicidades. Elaboración propia.

En esta versión rehíce el modelo con la notación de clases conceptuales de UML (nombre y atributos, sin tipos ni métodos, porque todavía describe el negocio y no el software) y agregué la asociación entre Usuario y Seguimiento para saber quién registró cada cambio. La barra en /puntaje y /severidad marca atributos derivados, que se calculan a partir del vector CVSS y no se cargan. Un cliente puede tener varios proyectos y cada proyecto tiene un solo cliente. El proyecto de un hallazgo se obtiene a través de su activo, por eso un activo con hallazgos no puede pasar a otro proyecto (cambiaría quién ve esa información).

5. Propuesta de solución

5.1 Propuesta funcional

La propuesta sigue el recorrido de un hallazgo, desde la preparación de la estructura hasta la consulta del resultado, y la tabla ordena las funciones en ese mismo orden.

Momento	Funciones del sistema	CU
1. Preparación	El administrador crea las cuentas, los proyectos con su cliente, los activos y el catálogo de responsables.	CU10, CU09, CU01, CU02
2. Registro	Alta del hallazgo en estado PENDIENTE. La severidad se calcula en el mismo formulario o queda «Sin evaluar» para después.	CU03, CU12
3. Tratamiento	Asignación de responsable y fecha objetivo y cambios de estado con comentario, que quedan en el historial.	CU04, CU06, CU07
4. Consulta	Listados por proyecto, estado y severidad. El cliente solo ve sus proyectos y el administrador puede exportar el resultado.	CU05, CU13, CU08
Transversal	Inicio y cierre de sesión, que habilita las funciones según el rol.	CU11

El administrador usa todas las funciones y el cliente tiene tres (iniciar sesión, consultar sus hallazgos y ver el historial). Ninguna consulta del cliente muestra datos de un proyecto que no le pertenece, aunque conozca el número de un hallazgo ajeno.

5.2 Propuesta técnica

La aplicación es de escritorio y sigue el patrón modelo-vista-controlador (MVC) con una capa DAO para el acceso a datos. Las vistas Swing muestran y capturan datos, los controladores coordinan cada caso de uso, las clases del modelo aplican las reglas (transiciones, fechas y cálculo CVSS) y los DAO ejecutan las consultas con JDBC. Tener el acceso a datos aparte deja cambiar la ubicación de la base sin tocar las pantallas.

Componente	Elección	Motivo
Lenguaje	Java 21	Lo pide la materia y es una versión con soporte extendido.

Componente	Elección	Motivo
Interfaz	Swing	Viene incluida en el JDK y no necesita librerías externas.
Base de datos	MySQL de XAMPP (MariaDB 10.4)	La pide la materia y tiene claves foráneas y transacciones (motor InnoDB).
Acceso a datos	JDBC con MySQL Connector/J	Es la API estándar de Java y las sentencias preparadas evitan la inyección SQL.
IDE	Apache NetBeans	Tiene diseñador de formularios Swing y depurador.
Diagramas	PlantUML	Los diagramas quedan guardados como texto en el repositorio.
Versionado	Git y GitHub	Historial de cambios y publicación del código.
Hardware	PC con Windows 10 u 11, 8 GB de RAM y red local	Alcanza para el JRE y el servidor de base en el mismo equipo del prototipo.

6. Requerimientos

6.1 Requerimientos funcionales

Cada requerimiento se corresponde con el caso de uso del mismo número.

RF / CU	Descripción
RF01 / CU01 Gestionar activos	El administrador da de alta, modifica y lista activos de un proyecto. Solo puede borrar o trasladar un activo que no tenga vulnerabilidades.
RF02 / CU02 Gestionar responsables	El administrador da de alta, modifica y lista responsables. Solo puede borrar uno que no tenga vulnerabilidades asignadas.
RF03 / CU03 Registrar vulnerabilidad	El administrador registra título, descripción, tipo, elemento afectado, activo y fecha de detección. El sistema asigna un número y el estado PENDIENTE.
RF04 / CU04 Asignar responsable	El administrador asigna un responsable y una fecha objetivo igual o posterior a la detección, y puede cambiarlos mientras el hallazgo no esté cerrado.
RF05 / CU05 Consultar vulnerabilidades	El administrador consulta todos los proyectos y el cliente solo los suyos, con filtros de proyecto, estado y severidad (incluida «Sin evaluar»).
RF06 / CU06 Cambiar estado	El administrador inicia el tratamiento, cierra o reabre un hallazgo. El sistema valida la transición, exige un comentario y guarda estado y seguimiento en una misma operación.
RF07 / CU07 Consultar historial	El usuario consulta fecha y hora, estados anterior y nuevo, autor y comentario de cada cambio, con los mismos permisos por proyecto que en CU05.
RF08 / CU08 Exportar listado	El administrador guarda el resultado filtrado en un archivo TXT nuevo. El sistema informa si se guardó o si hubo un error.
RF09 / CU09 Gestionar proyectos	El administrador crea y modifica proyectos y les asigna una cuenta cliente activa. Solo puede borrar un proyecto sin activos.
RF10 / CU10 Gestionar usuarios	El administrador crea cuentas con nombre de acceso único, contraseña y rol, y puede cambiar contraseñas o desactivar cuentas (salvo al último administrador activo).
RF11 / CU11 Iniciar y cerrar sesión	El sistema valida las credenciales de una cuenta activa, habilita las operaciones de su rol y al cerrar elimina la sesión y los datos visibles.
RF12 / CU12 Calcular CVSS	El administrador selecciona las ocho métricas base de CVSS 3.1 y el sistema calcula puntaje y severidad, los guarda con el hallazgo y deja recalcular.

Las vulnerabilidades no se borran sino que se conservan para consultar su historial, y pasan a CERRADA cuando se resuelven. La severidad se informa como ninguna (0,0), baja (0,1 a 3,9), media (4,0 a 6,9), alta (7,0 a 8,9) o crítica (9,0 a 10,0) según FIRST (2019a).

6.2 Requerimientos no funcionales

ID	Requerimiento y criterio de aceptación
RNF01 Plataforma	Aplicación de escritorio en Java 21 y Swing, con persistencia en MySQL mediante JDBC, para Windows 10 u 11.
RNF02 Integridad	Se rechazan campos obligatorios vacíos, referencias inexistentes, fechas objetivo anteriores a la detección y transiciones inválidas.
RNF03 Atomicidad	El cambio de estado y su seguimiento se guardan en la misma transacción. Si falla una escritura se revierte todo y el hallazgo conserva su estado.
RNF04 Acceso	Toda operación exige sesión y verifica rol y proyecto. Un cliente no puede consultar un hallazgo ajeno aunque conozca su número.
RNF05 Contraseñas	Se guardan con hash SHA-256 y una sal distinta por usuario, nunca en texto plano.
RNF06 Tiempo de respuesta	Con hasta 5.000 hallazgos cargados, las consultas de CU05 y CU07 responden en menos de 2 segundos en el equipo de referencia.
RNF07 Usabilidad	Interfaz en español, campos obligatorios marcados, mensajes que dicen qué corregir y acciones visibles según el rol. Un usuario nuevo registra un hallazgo en menos de 3 minutos después de una demostración de 10 minutos.
RNF08 Normas	Los datos personales de las cuentas se tratan según la Ley 25.326 (datos mínimos, acceso restringido y confidencialidad) y la severidad sigue la especificación CVSS 3.1 de FIRST.

7. Casos de uso

7.1 Diagrama de casos de uso

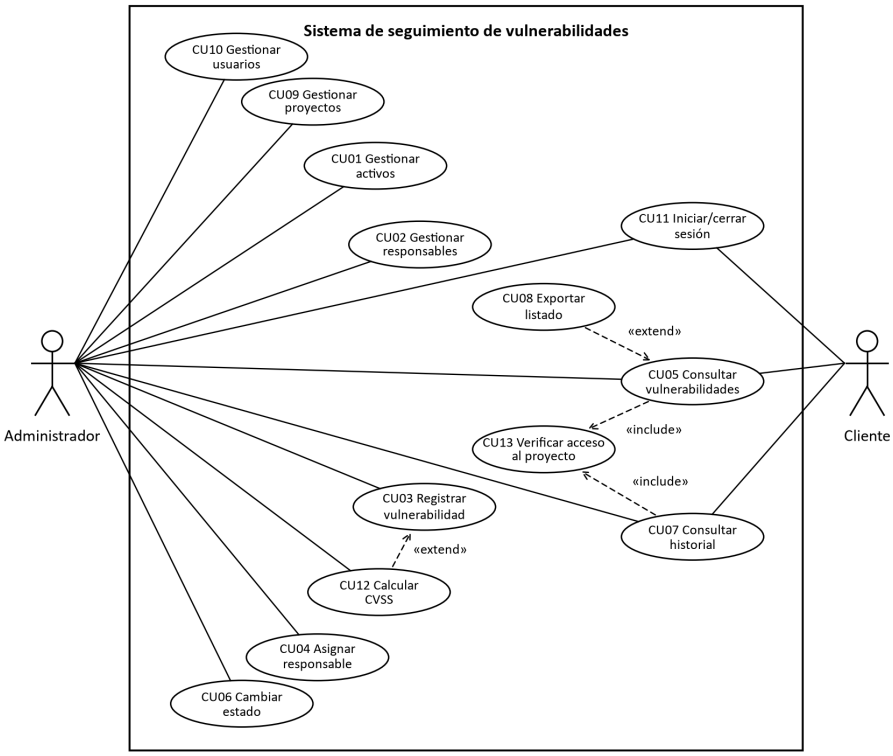


Figura 3. Diagrama de casos de uso con el límite del sistema. Elaboración propia.

El rectángulo marca el límite del sistema y deja a los actores afuera. Respecto de la primera entrega ordené el diagrama por actor (los casos del administrador a la izquierda y los compartidos junto al cliente) para que no se lea como una secuencia, y usé los estereotipos según su significado. «include» indica un comportamiento obligatorio que forma parte del caso base, por eso CU05 y CU07 incluyen CU13 (ninguna consulta puede mostrar datos sin verificar antes a qué proyectos accede la cuenta, y como la regla es la misma en los dos casos se escribe una sola vez). «extend» indica un comportamiento opcional que se agrega en un punto del caso base. CU12 extiende CU03 porque un hallazgo puede registrarse sin evaluar, y CU08 extiende CU05 porque la exportación es optativa y solo la usa el administrador. El inicio de sesión (CU11) no se modela como «include» de cada caso porque es una precondición de todos y no un paso de su flujo.

7.2 Especificación de los casos críticos

Detallo CU03, CU06 y CU11 porque cubren el registro, el seguimiento y el acceso, que son los puntos donde un error tiene más consecuencias (un cambio de estado sin historial o una sesión mal cerrada en un equipo compartido).

Caso de uso	**CU03 Registrar vulnerabilidad**
Actor	Administrador.
Objetivo	Incorporar un hallazgo asociado a un activo de un proyecto.
Requerimientos	RF03, RNF02 y RNF04.
Precondiciones	Sesión de administrador activa (CU11). Existen el proyecto y el activo.
Flujo principal	1. El administrador elige «Nuevo hallazgo». 2. Selecciona el proyecto y el activo. 3. Completa título, descripción, tipo, elemento afectado y fecha de detección. 4. Punto de extensión «evaluar severidad». 5. Confirma el alta. 6. El sistema valida los campos obligatorios, la fecha y que el activo pertenezca al proyecto. 7. Guarda el hallazgo con estado PENDIENTE. 8. Muestra el detalle.
Flujos alternativos	4a. Si el administrador completa las métricas se ejecuta CU12 y el vector queda guardado con el hallazgo. 6a. Si falta un dato o la fecha es inválida, el sistema indica qué campo corregir y vuelve al paso 3. 7a. Si falla la escritura informa el error y no se crea el hallazgo.
Postcondiciones	El hallazgo existe en estado PENDIENTE, sin responsable ni fecha objetivo, evaluado o «Sin evaluar», y con el historial vacío.

Caso de uso	**CU06 Cambiar estado**
Actor	Administrador.
Objetivo	Mover un hallazgo en su ciclo de estados y dejar constancia del motivo.
Requerimientos	RF06 (relacionado con RF04 y RF07), RNF02 y RNF03.
Precondiciones	Sesión de administrador activa y hallazgo existente. Para iniciar el tratamiento el hallazgo tiene responsable y fecha objetivo (CU04).
Flujo principal	1. El administrador selecciona un hallazgo y elige «Cambiar estado». 2. El sistema muestra el estado actual y solo las transiciones permitidas. 3. El administrador elige el nuevo estado y escribe el comentario. 4. El sistema valida la transición, la asignación y el comentario. 5. En una sola transacción actualiza el estado y registra el seguimiento (fecha y hora, usuario, estado anterior, estado nuevo y comentario). 6. Confirma y actualiza el listado.
Flujos alternativos	4a. Si la transición no está permitida (por ejemplo, cerrar desde PENDIENTE) se rechaza sin cambios. 4b. Si falta responsable o fecha objetivo se rechaza y se indica ejecutar CU04. 4c. Si el comentario está vacío se pide completarlo. 5a. Si falla una escritura se revierte la transacción y el hallazgo conserva su estado.
Postcondiciones	El estado actual coincide con el estado nuevo del último seguimiento.

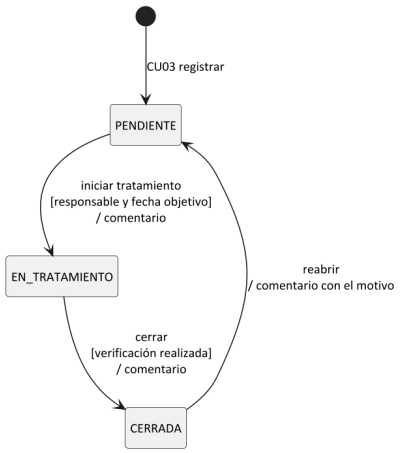


Figura 4. Estados de una vulnerabilidad y transiciones admitidas (CU06). Elaboración propia.

Caso de uso	**CU11 Iniciar y cerrar sesión**
Actores	Administrador y cliente.
Objetivo	Acceder con los permisos de la cuenta y cerrar el acceso al terminar.
Requerimientos	RF11, RNF04 y RNF05.
Precondiciones	La cuenta existe y está activa.
Flujo principal	1. El usuario ingresa nombre de acceso y contraseña. 2. El sistema busca la cuenta y calcula el hash de la contraseña con la sal guardada. 3. Si coincide, abre la sesión con el rol de la cuenta. 4. Muestra la consulta con las acciones de ese rol. 5. Al elegir «Cerrar sesión» descarta la sesión, limpia los listados y vuelve a la pantalla de acceso.
Flujos alternativos	2a. Si no hay conexión con la base se informa y no se abre la sesión. 3a. Si las credenciales son incorrectas o la cuenta está inactiva se muestra «Usuario o contraseña incorrectos» (el mismo mensaje en los dos casos, para no revelar qué cuentas existen).
Postcondiciones	Con sesión el usuario opera según su rol y sin sesión no hay operaciones disponibles.

8. Etapa de análisis

En el análisis los casos de uso se refinan en clases conceptuales con tres estereotipos. «boundary» representa la interacción con el actor, «control» coordina el caso de uso y «entity» guarda la información que persiste. Todavía no son clases de software (esas aparecen en el diseño) sino el paso intermedio entre los casos de uso y el diagrama de clases.

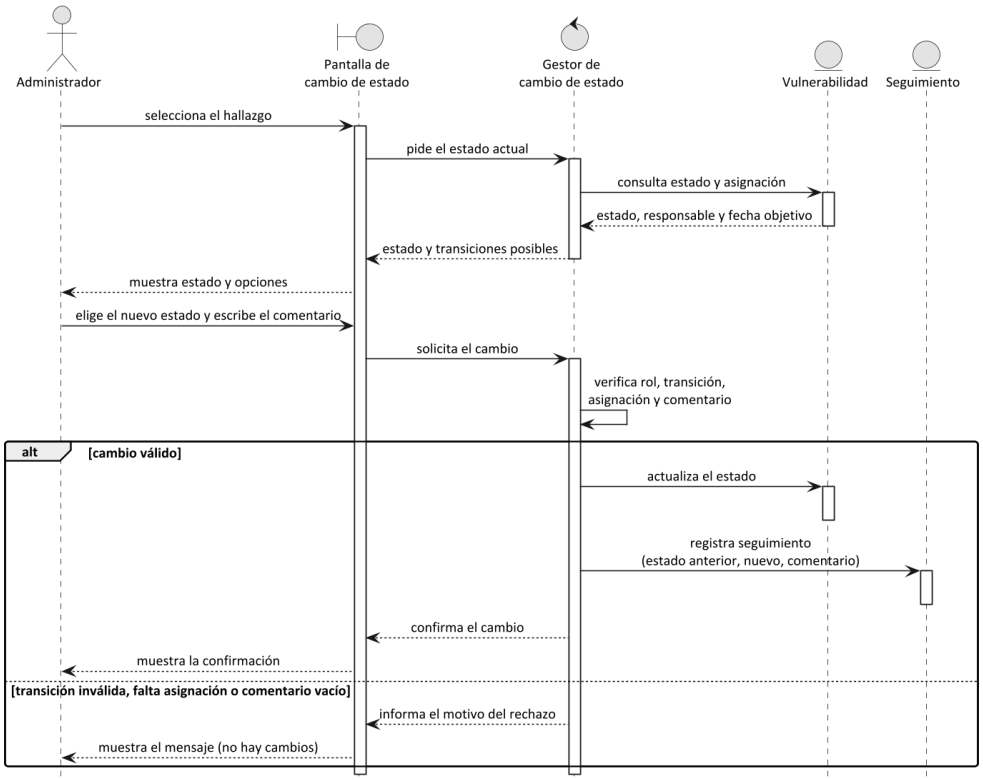


Figura 5. Diagrama de secuencia de análisis de CU06, curso normal y alternativo. Elaboración propia.

Elegí CU06 porque es el caso con más reglas y el que sostiene la trazabilidad. La pantalla solo recibe y muestra datos, y el gestor concentra las validaciones y es el único que modifica las entidades. El fragmento alt separa el curso normal, donde el estado y el seguimiento se escriben juntos, del curso alternativo, donde el gestor informa el motivo del rechazo y ninguna entidad cambia.

9. Etapa de diseño

En el diseño las clases de análisis pasan a ser clases de Java con atributos, métodos y tipos. Los atributos coinciden con las columnas de la base (int para las claves, LocalDate para las fechas y un enum para el estado), así los DAO convierten filas en objetos sin transformaciones intermedias. Omití los métodos get y set para que el diagrama se lea mejor.

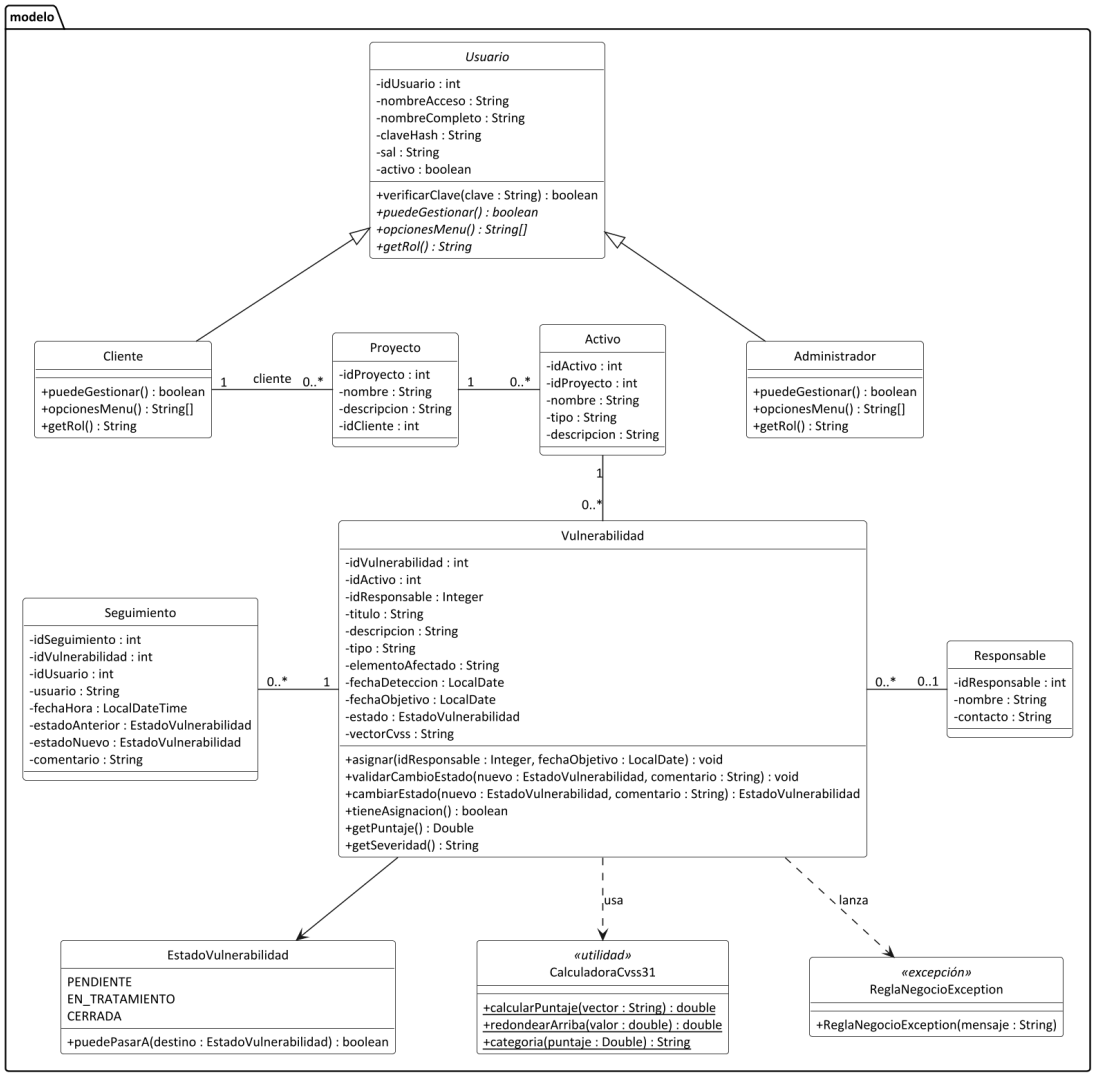


Figura 6. Diagrama de clases de diseño del paquete modelo, actualizado con el código final. Elaboración propia.

Usuario es abstracta y tiene dos subclases, Administrador y Cliente, que responden distinto a puedeGestionar() (la columna rol de la base decide qué subclase se crea al iniciar sesión). Vulnerabilidad concentra las reglas del ciclo de estados y EstadoVulnerabilidad define las tres transiciones permitidas. idResponsable es Integer y no int porque puede ser nulo. CalculadoraCvss31 es una clase utilitaria con métodos estáticos que aplica la fórmula de la especificación a partir del vector, y getPuntaje() la usa para devolver el puntaje (nulo si el hallazgo no está evaluado). En esta entrega corregí el diagrama para que coincida con lo implementado. Sumé los métodos que aparecieron al programar (opcionesMenu(), getRol(), tieneAsignacion() y getSeveridad()), el nombre de acceso que guarda Seguimiento para mostrar el autor y ReglaNegocioException, que es la excepción del modelo. Las vistas Swing, el controlador y los DAO que completan el MVC están en la figura de la sección 16.1.

10. Etapa de implementación

La implementación transforma las clases de diseño en código y organiza su compilación, integración y ejecución. En la versión final el código queda en seis paquetes (modelo con las entidades y sus reglas, vista con las ventanas Swing, controlador con la coordinación de los casos de uso y los permisos, dao con el acceso a MySQL por JDBC, util con los archivos de texto y app con el arranque). Así las entidades no administran la interfaz ni el almacenamiento. La única dependencia externa es el driver MySQL Connector/J 8.4.0, que va en la carpeta lib junto al archivo .jar del sistema.

La lectura 2 define el modelo de despliegue como la distribución física del sistema entre los nodos de cómputo (*Proceso unificado de desarrollo*, s. f.). Las decisiones de despliegue salen de los requerimientos no funcionales. El esquema siguiente es el de la versión final.

RNF	Decisión de despliegue
RNF01	JRE 21 en cada PC con Windows. La aplicación se distribuye como un único archivo .jar.
RNF03	Base MySQL con motor InnoDB, que maneja transacciones y claves foráneas.
RNF04 y RNF08	La base queda en un equipo de la red interna, sin salida a Internet, y el puerto 3306 solo se habilita para las PC del área.
RNF06	El servidor de base está en la misma red local que los puestos que consultan, sin pasar por Internet.

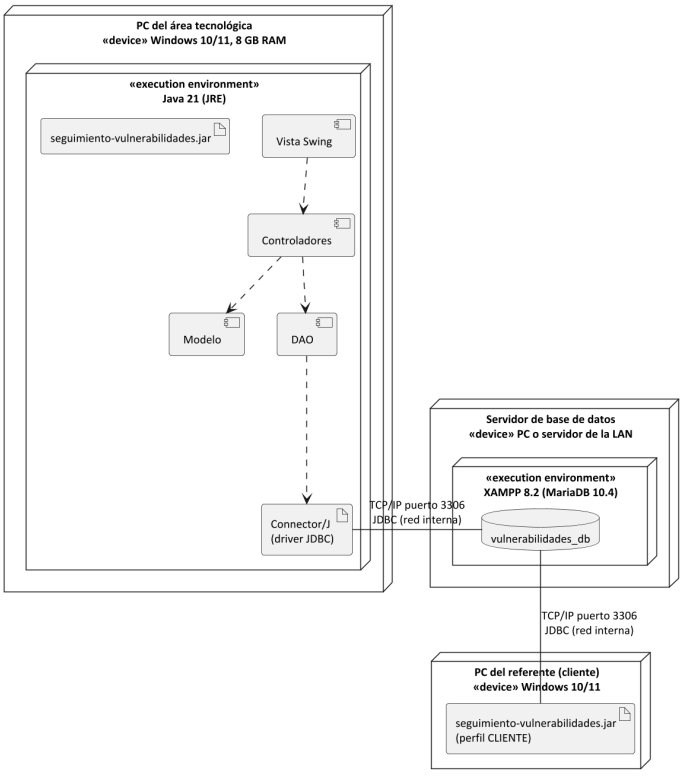


Figura 7. Diagrama de despliegue. Elaboración propia.

En las pruebas de la versión final los tres nodos estuvieron en el mismo equipo (localhost). En la operación real la base queda en un equipo del área y las PC del administrador y de los referentes ejecutan el mismo .jar (lo único que cambia es el rol de la cuenta que inicia sesión). Hay un riesgo que dejo documentado. Con una conexión directa a la base, cada PC que ejecuta la aplicación tiene la cadena de conexión, por eso el acceso por proyecto se controla también en las consultas y en producción se usaría un usuario de base con permisos mínimos (la aplicación ya permite indicar otro usuario al ejecutarla). Los scripts y las pruebas usaron el usuario local de XAMPP.

11. Etapa de pruebas

La lectura 2 organiza las pruebas en unitarias (de caja blanca y de caja negra), de integración, de sistema y de aceptación, y aclara que buscan que el software falle antes de que lo haga frente al usuario (*Proceso unificado de desarrollo*, s. f.).

11.1 Plan de pruebas

CU	Código	Tipo	Técnica	Observaciones	Estado
CU12	CP01	Unitaria	Caja negra, valores límite	CalculadoraCvss31.categoria()	Ejecutada
CU12	CP02	Unitaria	Caja negra, ejemplos oficiales	calcularPuntaje() con tres vectores de FIRST	Ejecutada
CU06	CP03	Unitaria	Caja blanca, caminos básicos	Vulnerabilidad.validarCambioEstado()	Ejecutada
CU03, CU04	CP04	Integración	Caja negra, datos inválidos	CHECK de fechas y clave foránea de la base	Ejecutada
CU05, CU13	CP05	Sistema	Funcional	Un cliente no ve hallazgos ajenos desde la interfaz	Ejecutada (TP4)
CU03, CU12	CP07	Integración	Caja negra	Alta desde la ventana guardada en MySQL	Ejecutada (TP4)
CU06, CU04	CP08	Integración	Caja negra, error forzado	Transacción del cambio de estado y rollback	Ejecutada (TP4)
CU08	CP09	Integración	Caja negra, datos inválidos	Exportación TXT e importación CSV	Ejecutada (TP4)
CU11	CP10	Sistema	Caja negra	Inicio de sesión con el servidor de base apagado	Ejecutada (TP4)
Todos	CP06	Aceptación	Validación con el usuario	Recorrido completo de un hallazgo con el usuario de referencia	Planificada

11.2 Caso de prueba de caja negra (CP01)

Verifica RF12 sobre CalculadoraCvss31.categoria(), que recibe el puntaje y devuelve la severidad. Las clases válidas son null (sin evaluar), 0,0 (ninguna), 0,1 a 3,9 (baja), 4,0 a 6,9 (media), 7,0 a 8,9 (alta) y 9,0 a 10,0 (crítica), y las inválidas son los puntajes menores que 0,0 o mayores que 10,0. Sobre cada frontera tomé un valor de cada lado (análisis de valores límite).

Entrada	Esperado	Obtenido	Entrada	Esperado	Obtenido
null	Sin evaluar	Sin evaluar	6,9	Media	Media
-0,1	Rechazo	Rechazo	7,0	Alta	Alta
0,0	Ninguna	Ninguna	8,9	Alta	Alta
0,1	Baja	Baja	9,0	Crítica	Crítica
3,9	Baja	Baja	10,0	Crítica	Crítica
4,0	Media	Media	10,1	Rechazo	Rechazo

Los 12 casos dieron el resultado esperado (salida_pruebas_java.txt del repositorio). CP02 completa la prueba del cálculo comparando calcularPuntaje() con tres vectores publicados por FIRST (2019b), 6,1 para CVE-2013-1937, 7,5 para Heartbleed y 9,8 para Shellshock, y los tres coincidieron.

11.3 Caso de prueba de caja blanca (CP03)

Verifica el método validarCambioEstado() de Vulnerabilidad, que tiene tres decisiones (D1 transición no permitida, D2 inicio del tratamiento sin asignación y D3 comentario vacío). El grafo de flujo tiene 9 nodos y 11 aristas, así que la complejidad ciclomática es V(G) = 11 - 9 + 2 = 4 y hay cuatro caminos independientes.

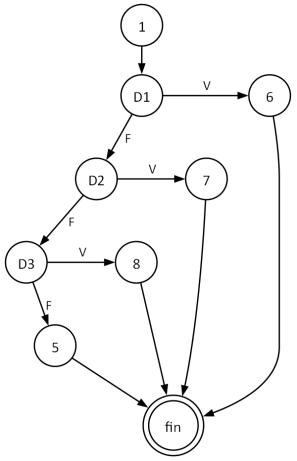


Figura 8. Grafo de flujo de validarCambioEstado(). Elaboración propia.

Camino	Recorrido	Datos de entrada	Esperado	Obtenido
C1	1, D1, 6, fin	PENDIENTE a CERRADA	Rechazo (transición)	Rechazo
C2	1, D1, D2, 7, fin	Inicio sin responsable ni fecha	Rechazo (asignación)	Rechazo
C3	1, D1, D2, D3, 8, fin	Asignado, comentario en blanco	Rechazo (comentario)	Rechazo
C4	1, D1, D2, D3, 5, fin	Asignado, comentario válido	EN_TRATAMIENTO	EN_TRATAMIENTO

11.4 Procedimiento de prueba

1. Compilar el modelo y las pruebas con el JDK 21 desde la carpeta TP2/pruebas del repositorio (javac -encoding UTF-8 -d bin src/modelo/*.java src/pruebas/*.java).
2. Ejecutar java -cp bin pruebas.PruebasTP2 y guardar la salida. Cada caso imprime PASA o FALLA con el valor esperado y el obtenido.
3. Recrear la base con los scripts 01 a 04 y revisar que la base rechace los dos datos inválidos de CP04.
4. Si un resultado no coincide, registrar el defecto con la plantilla de 11.5 y, después de corregirlo, repetir las pruebas afectadas antes de cerrarlo.

11.5 Tratamiento de defectos

Cada defecto sigue el ciclo abierto, asignado, corregido, verificado y cerrado, y si la verificación falla vuelve a asignado (es el mismo ciclo que el sistema propone para los hallazgos). Como las pruebas ejecutadas en esta etapa no encontraron defectos, la plantilla se muestra completa con un defecto de ejemplo.

Campo	Registro
Identificador	DEF-01 (ejemplo)
Prueba de origen	CP01, caso categoria(0.0)
Descripción	Devuelve «Baja» cuando el resultado esperado es «Ninguna».
Severidad	Media (un hallazgo sin impacto aparecería como de riesgo bajo).

Campo	Registro
Causa	La comparación puntaje < 4.0 se evalúa antes que el caso 0,0.
Acción	Evaluar primero el caso 0,0 y después los rangos.
Verificación	Repetir CP01 completo (regresión) y cerrar el defecto si pasan los 12 casos.
Estado	Cerrado

11.6 Evaluación de las pruebas

Prueba	Casos	Aprobados	Fallidos	Observación
CP01	12	12	0	Todas las fronteras de las categorías de severidad.
CP02	3	3	0	Coincide con los ejemplos oficiales de FIRST.
CP03	4	4	0	Cubre los cuatro caminos independientes.
CP04	2	2	0	La base rechazó la fecha inválida (4025) y el borrado con hallazgos (1451).
CP05, CP07 a CP10	17	17	0	Verificaciones sobre la interfaz y la base (sección 16.7).
Total	38	38	0	Sin defectos abiertos.

CP01 a CP04 se ejecutaron en el TP2. En esta entrega se ejecutó CP05, que había quedado planificada, y se sumaron CP07 a CP10 sobre la versión final (el detalle está en la sección 16.7). CP06 sigue planificada porque necesita al usuario de referencia, y el rendimiento de RNF06 con 5.000 hallazgos todavía no se midió.

12. Prototipos de interfaz

Los prototipos son ventanas Swing reales (el mismo toolkit del sistema) generadas por el programa PrototiposInterfaz del repositorio. No tienen lógica conectada, salvo el panel CVSS, que usa la clase CalculadoraCvss31. Siguiendo a Kendall y Kendall (2011), sirven para mostrar cómo se va a ver la herramienta antes de construirla, y cada uno está vinculado al caso de uso que implementa.

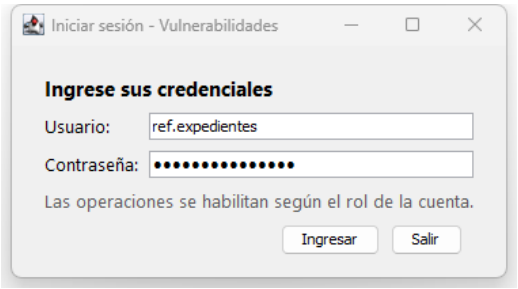


Figura 9. Prototipo de inicio de sesión (CU11). Elaboración propia.

Es la pantalla inicial de CU11, con nombre de acceso y contraseña enmascarada.

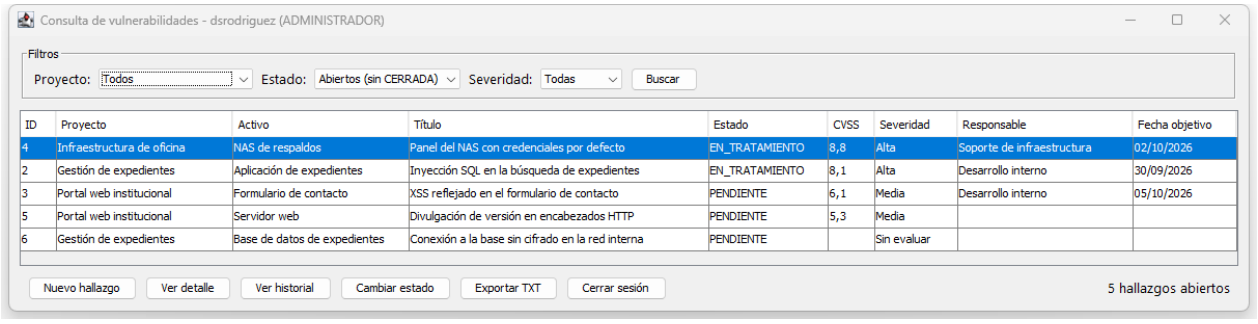


Figura 10. Prototipo de consulta de vulnerabilidades (CU05). Elaboración propia.

Es la pantalla principal. Los filtros son los de RF05 y los botones abren el registro, el historial, el cambio de estado y la exportación. Para una cuenta cliente se ocultan los botones de gestión y el filtro de proyecto solo lista sus proyectos (CU13).

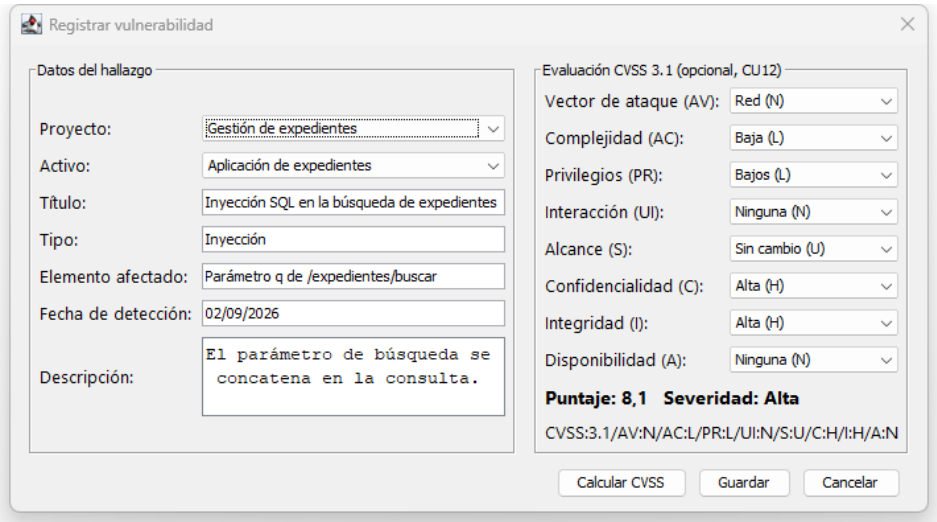


Figura 11. Prototipo de registro de vulnerabilidad con el panel CVSS (CU03 extendido por CU12). Elaboración propia.

El formulario de la izquierda es CU03 y el panel de la derecha es la extensión CU12. Si no se completan las métricas el hallazgo queda «Sin evaluar». El puntaje que se ve (8,1, alta) lo calculó la clase real. Las ventanas de la versión final mantienen estos tres diseños (sección 16).

13. Base de datos

13.1 Definición y criterios de diseño

La base se llama vulnerabilidades_db y corre en el MySQL de XAMPP (MariaDB 10.4, compatible con MySQL). El criterio principal fue evitar la redundancia (Reinosa et al., 2012, como se citó en *Proceso unificado de desarrollo*, s. f.) sin complicar el esquema. Quedaron seis tablas con claves primarias numéricas autoincrementales, una clave foránea por cada relación, UNIQUE en los datos que no se pueden repetir (nombre de acceso y nombre de proyecto), ENUM para el rol y el estado, y una restricción CHECK para que la fecha objetivo no sea anterior a la detección. Las reglas más complejas (transiciones de estado y permisos por proyecto) se controlan en Java. El motor es InnoDB porque maneja transacciones y claves foráneas, y el juego de caracteres utf8mb4 guarda sin problemas tildes y eñes.

13.2 Normalización

Todos los atributos son atómicos y todas las claves primarias son simples, por lo que las tablas están en primera y segunda forma normal. Para la tercera forma normal ningún atributo no clave depende de otro atributo no clave, y por eso el puntaje y la severidad no se guardan (dependen del vector CVSS y se calculan en Java cada vez que hacen falta, así nunca quedan desactualizados). El vector se guarda como una sola cadena porque es el formato estándar de FIRST y la aplicación siempre lo lee y lo escribe completo.

13.3 Diagrama entidad-relación

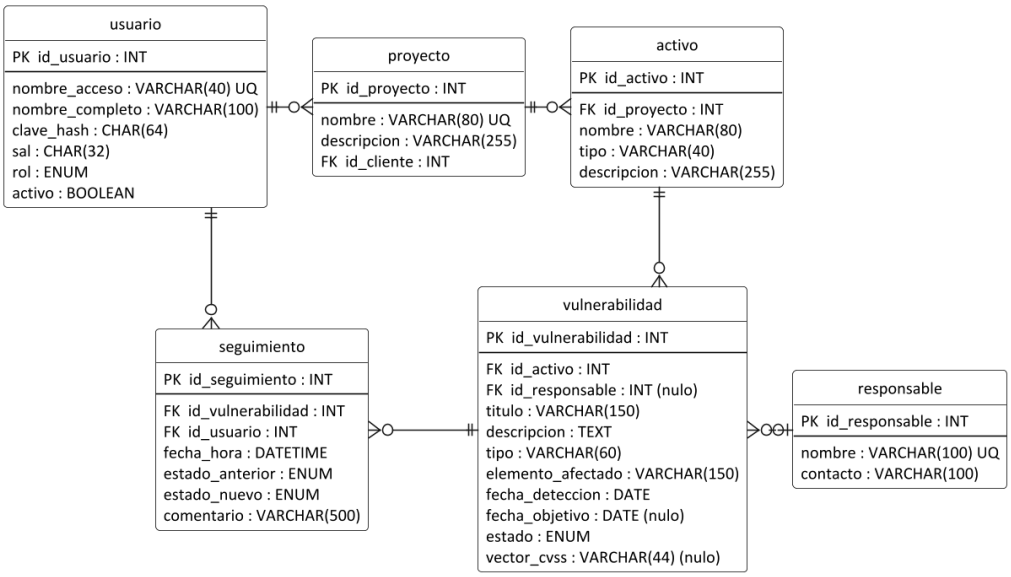


Figura 12. Diagrama entidad-relación (notación pata de gallo) con claves y tipos. Elaboración propia.

En la notación pata de gallo la raya indica uno, el círculo cero (opcional) y la pata muchos. Un hallazgo tiene cero o un responsable (id_responsable admite nulo) y un responsable puede tener muchos hallazgos. El usuario aparece en dos relaciones, como cliente de proyectos y como autor de seguimientos.

13.4 Creación de la base y de las tablas

El script 01_creacion.sql crea la base con utf8mb4 y las seis tablas. La captura muestra el resultado en phpMyAdmin, todavía sin registros.

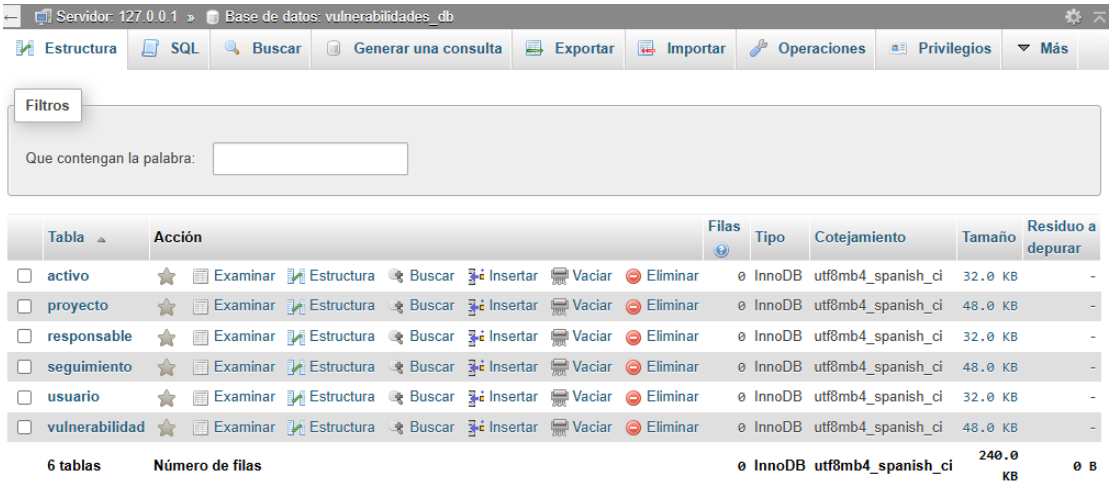


Figura 13. Base vulnerabilidades_db con sus seis tablas en phpMyAdmin. Captura propia.

13.5 Inserción de registros

El script 02_insercion.sql carga datos de prueba ficticios (tres cuentas, tres proyectos, seis activos, cuatro responsables, seis hallazgos en distintos estados y seis seguimientos, uno con reapertura). Ningún dato corresponde a hallazgos reales de la organización, porque publicar vulnerabilidades reales en un repositorio abierto sería justamente la fuga de información que el sistema busca evitar.

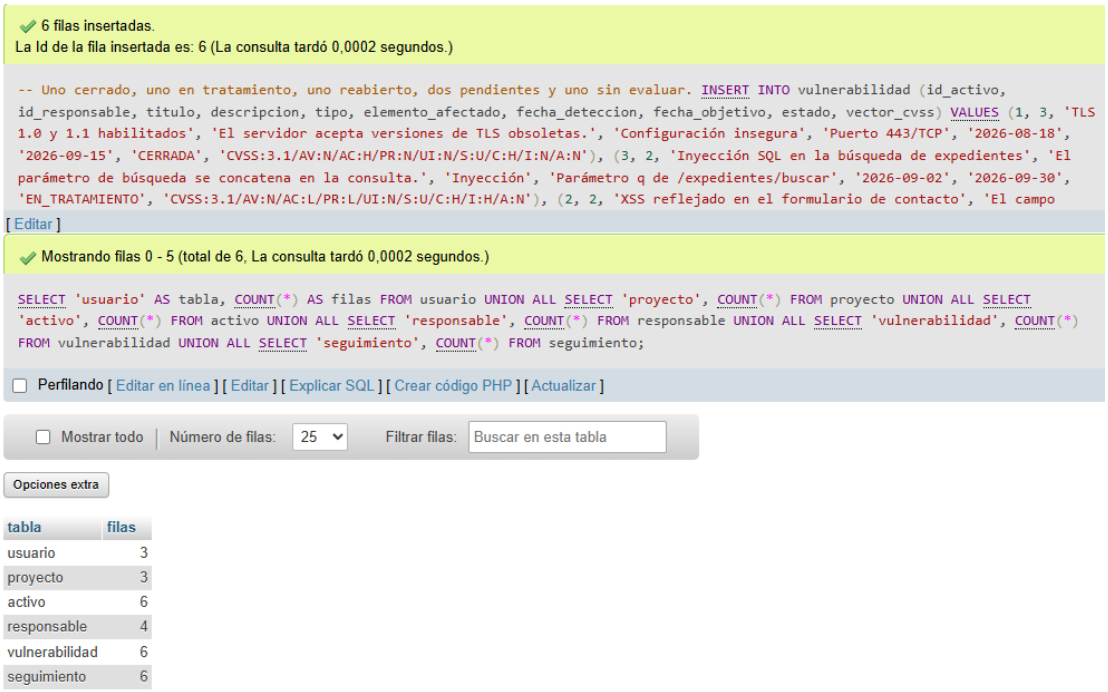


Figura 14. Ejecución del script de inserción en phpMyAdmin y cantidad de filas por tabla. Captura propia.

13.6 Consulta de registros

La consulta principal del administrador (CU05) une vulnerabilidad, activo y proyecto, y usa LEFT JOIN con responsable porque un hallazgo puede no tener responsable todavía.

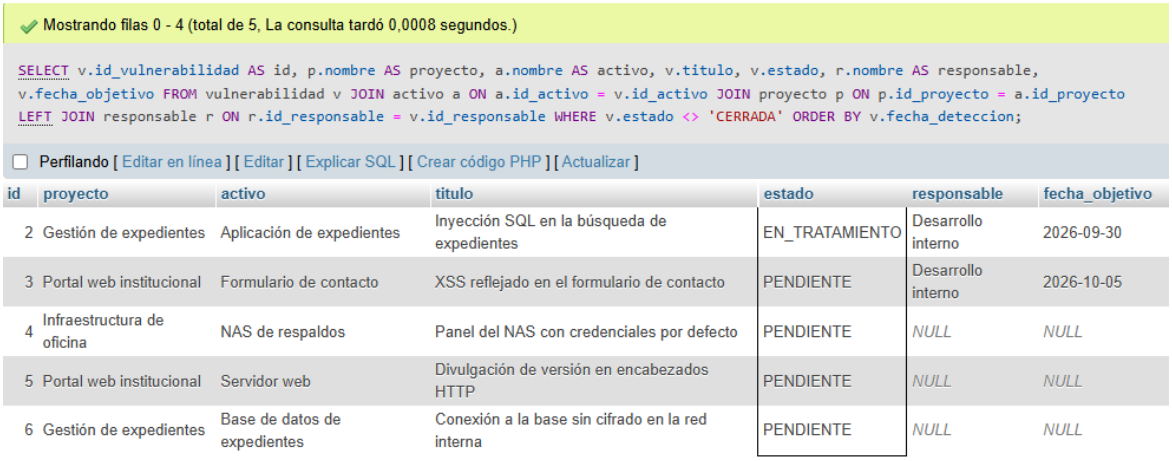


Figura 15. Consulta de hallazgos abiertos con JOIN en phpMyAdmin. Captura propia.

El script 03_consultas.sql tiene además el listado del cliente (para el cliente 3 devuelve solo los hallazgos 2, 4 y 6, que son de sus dos proyectos), el historial del hallazgo 3 con su reapertura, la cantidad de hallazgos por proyecto y estado, y la asignación del hallazgo 4 con el inicio de su tratamiento dentro de una transacción (UPDATE del hallazgo e INSERT del seguimiento con un único COMMIT). Al final intenta poner una fecha objetivo anterior a la detección y la base lo rechaza con el error 4025 de la restricción ck_fecha_objetivo.

13.7 Borrado de registros

El script 04_borrado.sql borra un responsable sin hallazgos, que se permite, e intenta borrar un activo que sí tiene. La clave foránea lo impide con el error 1451, que es lo que protege el historial (si se pudiera borrar, sus hallazgos quedarían apuntando a un activo que no existe). Por último borra un activo sin hallazgos y esa operación se completa.

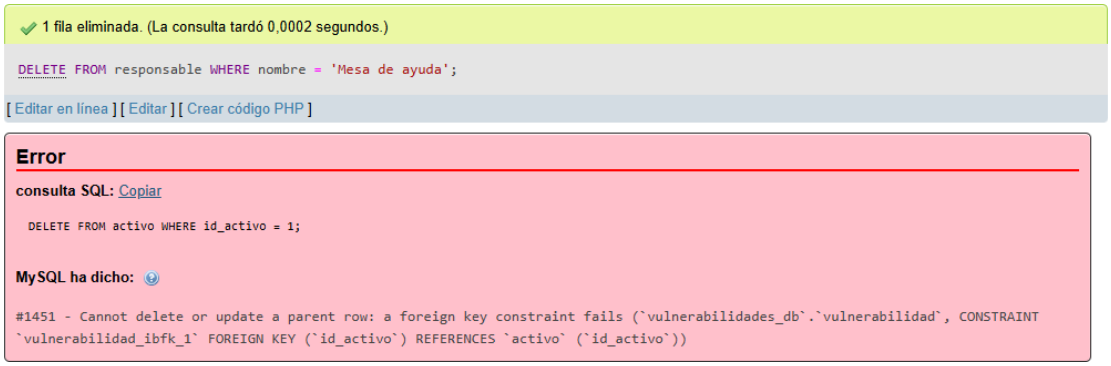


Figura 16. Borrado permitido y borrado rechazado por la clave foránea (error 1451) en phpMyAdmin. Captura propia.

13.8 Presentación de las consultas SQL

Siguiendo la indicación de no pegar todas las consultas en el informe, los scripts completos están en la carpeta TP2/sql del repositorio (https://github.com/rodriguezd0/semprac-seguimiento-vulnerabilidades), numerados en el orden de ejecución, y la salida real de cada uno está en TP2/salidas.

Archivo	Contenido
01_creacion.sql	Base con utf8mb4 y las seis tablas con sus claves y restricciones.
02_insercion.sql	Datos de prueba ficticios y cantidad de filas por tabla.
03_consultas.sql	Consultas de CU05, CU07 y CU13, resumen por estado, transacción de CU04 y CU06 y prueba del CHECK.
04_borrado.sql	Borrados permitidos y borrado rechazado por la clave foránea.

14. Definiciones de comunicación

La comunicación de la versión final ocurre entre la aplicación Java y el servidor de base de datos, sin intercambio con sistemas externos. Usa JDBC con el protocolo cliente-servidor de MySQL sobre TCP, en el puerto 3306. En las pruebas los dos extremos estuvieron en el mismo equipo y en la operación real el tráfico viaja por la red interna del área.

Capa	Definición en el proyecto
Aplicación	JDBC con MySQL Connector/J y protocolo de MySQL, con sentencias preparadas.
Transporte	TCP en el puerto 3306 del servidor (conexión orientada, con entrega ordenada y confirmada).
Red	IPv4 en la subred privada de la oficina, sin publicación hacia Internet.
Enlace y física	Ethernet (IEEE 802.3) con cable UTP categoría 5e o 6 hasta el switch del área, y Wi-Fi (IEEE 802.11) solo para equipos de consulta.

El firewall del servidor filtra el puerto 3306 para que solo lo alcancen las PC del área. Como mejora queda habilitar TLS en la conexión JDBC para cifrar el tráfico dentro de la red (de hecho, uno de los hallazgos de prueba es una base que acepta conexiones sin cifrar).

15. Desarrollo del prototipo en Java

Esta sección documenta el incremento del TP3 tal como se entregó. La versión final, con Swing y MySQL, está en la sección 16 y reutiliza las clases del paquete modelo. En ellas solo cambiaron los botones de cada rol (16.4) y se sumaron controles de largo y de formato de los datos (16.7).

Kendall y Kendall (2011) definen el prototipo como un modelo operacional que incluye solo algunas características del sistema final. Para esta etapa armé un prototipo de consola con los datos en memoria,

porque lo primero que quería probar era el modelo del diseño (sobre todo el ciclo de estados y el acceso por proyecto, ya que el cálculo CVSS venía probado del TP2) sin depender todavía de la interfaz ni de la base. Cubre el inicio de sesión (CU11), la consulta según el rol (CU05 con CU13), el registro con cálculo CVSS (CU03 con CU12), la asignación (CU04), el cambio de estado (CU06) y el historial (CU07). Los datos iniciales son los del script 02_insercion.sql, así el prototipo muestra los mismos hallazgos que la base y acepta las mismas claves de prueba.

La lectura 3 resuelve el caso ECOViento con cuatro clases del dominio, una clase con el menú y una excepción propia (*Programación orientada a objetos*, s. f.). Seguí esa misma idea, con las clases del diagrama de diseño de la sección 9 en el paquete modelo y dos clases en el paquete app.

15.1 Estructura del prototipo

Son 13 clases y unas 1.230 líneas con comentarios, de las cuales 132 son la calculadora CVSS que viene del TP2 y unas 65 son los datos de prueba.

Paquete	Clase	Responsabilidad
modelo	Usuario (abstracta)	Datos de la cuenta y verificarClave() con SHA-256 de sal más clave. Declara puedeGestionar(), opcionesMenu() y getRol().
modelo	Administrador y Cliente	Subclases de Usuario, cada una con sus permisos y su menú.
modelo	Proyecto, Activo y Responsable	Catálogo con atributos privados, constructor y métodos get.
modelo	Vulnerabilidad	Hallazgo con asignar(), validarCambioEstado(), cambiarEstado(), getPuntaje() y getSeveridad().
modelo	EstadoVulnerabilidad	Enumeración de los tres estados con puedePasarA().
modelo	Seguimiento	Un cambio del historial, que una vez creado no se modifica.
modelo	CalculadoraCvss31	Cálculo CVSS 3.1 base. Viene del TP2 sin cambios (ya pasó CP01 y CP02).
modelo	ReglaNegocioException	Excepción propia para las reglas del negocio.
app	SistemaVulnerabilidades	Un ArrayList por entidad con los datos de prueba, los casos de uso del prototipo, el ordenamiento y la búsqueda.
app	MenuConsola	main(), lectura con Scanner, menú de selección y captura de excepciones.

15.2 Características pedidas por la consigna y dónde están en el código

Característica	Dónde está	Por qué
Sintaxis y tipos de datos	int y String en los catálogos, Integer y Double en Vulnerabilidad, LocalDate para las fechas y un enum para el estado.	Integer y Double admiten null, que es como se representa «sin responsable» y «sin evaluar» (igual que las columnas nulas de la base).
Estructuras condicionales	if y else en las reglas de Vulnerabilidad y en puedeVer(), switch con flechas en MenuConsola.ejecutar().	Cada regla del CU06 es una decisión (las mismas D1, D2 y D3 del caso de caja blanca CP03).
Estructuras repetitivas	for en el menú y en los listados, for-each en las búsquedas, while en la búsqueda binaria y en el inicio de sesión, do-while en el menú y en las métricas CVSS.	do-while muestra el menú al menos una vez y vuelve a pedir una métrica hasta que el valor sea válido.
Estructuras de datos	Un ArrayList por entidad y arreglos para las métricas CVSS (METRICAS, NOMBRES y la matriz VALORES).	La cantidad de hallazgos cambia al registrar, en cambio las métricas son siempre ocho.

Característica	Dónde está	Por qué
Excepciones	throw de ReglaNegocioException en el modelo y try con cuatro catch en MenuConsola.menu().	Un dato mal cargado se informa y el menú sigue, el programa no se corta.
Encapsulamiento	Atributos private con métodos get. El estado solo cambia con cambiarEstado() y las listas salen como copias.	Nadie puede pasar un hallazgo a CERRADA salteando las validaciones.
Herencia	Administrador y Cliente extienden Usuario y llaman a super(...).	Los datos de la cuenta y verificarClave() se escriben una sola vez.
Polimorfismo	puedeGestionar(), opcionesMenu() y getRol() sobrescritos en cada subclase. El menú usa una variable de tipo Usuario.	El menú no pregunta el rol, cada objeto responde con su versión.
Abstracción	Usuario es abstract y declara tres métodos abstractos.	No existe una cuenta sin rol, por eso Usuario no se puede instanciar.
Menú de selección	MenuConsola.menu() y leerOpcion().	Arma las opciones con el arreglo de opcionesMenu() y rechaza lo que no sea una opción válida.
Objetos y constructores	new en cargarDatosPrueba() y en registrar(). Vulnerabilidad tiene dos constructores y el corto llama al completo con this(...).	El constructor valida los datos obligatorios, así no se crea un hallazgo sin título.
Ordenación (opcional)	ordenarPorPuntaje(), por inserción.	El puntaje se calcula en Java (sección 13.2), por eso el orden también.
Búsqueda (opcional)	buscarPorNumero(), búsqueda binaria.	La lista está ordenada por número porque cada hallazgo nuevo va al final.

15.3 Compilación y ejecución

Se necesita el JDK 21 y no hay librerías externas. Desde la carpeta TP3 del repositorio se compila y se ejecuta con los dos comandos que siguen (en NetBeans alcanza con crear un proyecto con la carpeta src y ejecutar app.MenuConsola).

```
javac -encoding UTF-8 -d bin src/modelo/*.java src/app/*.java
java -cp bin app.MenuConsola
```

La compilación con todas las advertencias de javac activadas (opción Xlint con el valor all) terminó sin errores ni advertencias y generó las 13 clases (salida_00_compilacion.txt). Para dejar evidencia armé tres sesiones con la entrada guionada en archivos (administrador, cliente y errores de carga) y las ejecuté con la opción -eco (con ella el programa repite cada línea que lee, así la salida guardada se entiende sola).

```
java -Dstdout.encoding=UTF-8 -cp bin app.MenuConsola -eco \
     < entradas/entrada_01_administrador.txt > salidas/salida_01_administrador.txt
```

15.4 Fragmentos de código

Los fragmentos se muestran sin los comentarios, que están completos en el repositorio. Usuario es abstracta y declara los métodos que cada rol resuelve a su manera. Cliente la extiende, inicializa lo heredado con super(...) y sobrescribe opcionesMenu(). El menú trabaja con una variable de tipo Usuario y no necesita saber si del otro lado hay un administrador o un cliente.

```
public abstract class Usuario {
    ...
    public abstract boolean puedeGestionar();
    public abstract String[] opcionesMenu();
    public abstract String getRol();
}

public class Cliente extends Usuario {
    public Cliente(int idUsuario, String nombreAcceso, String nombreCompleto,
                   String claveHash, String sal, boolean activo) {
        super(idUsuario, nombreAcceso, nombreCompleto, claveHash, sal, activo);
    }
    @Override
    public String[] opcionesMenu() {
        return new String[] {"Listar mis vulnerabilidades", "Buscar por número",
                             "Ordenar por puntaje CVSS"};
    }
    ...
}

// En MenuConsola.menu()
String[] opciones = usuario.opcionesMenu();
...
for (int i = 0; i < opciones.length; i++) {
    System.out.println("  " + (i + 1) + ". " + opciones[i]);
}
```

Fragmento 1. Herencia, abstracción y polimorfismo en Usuario, Cliente y MenuConsola.

Las reglas del CU06 están en el modelo y se informan con throw. El menú las atrapa con try y catch junto con los errores de carga (un número mal escrito, una fecha imposible o un estado que no existe). NumberFormatException va antes que IllegalArgumentException porque es su subclase (al revés el programa ni compila, ya que el catch general la atraparía primero).

```
// En Vulnerabilidad.validarCambioEstado()
if (nuevo == null || !estado.puedePasarA(nuevo)) {
    throw new ReglaNegocioException("Transición no permitida de " + estado + " a " + nuevo + ".");
}
if (nuevo == EstadoVulnerabilidad.EN_TRATAMIENTO && !tieneAsignacion()) {
    throw new ReglaNegocioException("Falta responsable o fecha objetivo (primero hay que asignarlo).");
}

// En MenuConsola.menu()
try {
    ejecutar(usuario, opcion);
} catch (ReglaNegocioException e) {
    System.out.println("OPERACIÓN RECHAZADA. " + e.getMessage());
} catch (NumberFormatException e) {
    System.out.println("DATO INVÁLIDO. Se esperaba un número.");
} catch (DateTimeParseException e) {
    System.out.println("DATO INVÁLIDO. La fecha va como dd/mm/aaaa y tiene que existir.");
} catch (IllegalArgumentException e) {
    System.out.println("DATO INVÁLIDO. " + e.getMessage());
}
```

Fragmento 2. Lanzamiento y captura de excepciones.

El puntaje no se guarda en la base (sección 13.2) sino que se calcula en Java, así que el orden por puntaje también se hace en Java. Elegí el ordenamiento por inserción porque con pocos hallazgos alcanza y es estable (los empates quedan en el orden en que se cargaron).

```
public static ArrayList<Vulnerabilidad> ordenarPorPuntaje(ArrayList<Vulnerabilidad> origen) {
    ArrayList<Vulnerabilidad> lista = new ArrayList<>(origen);
    for (int i = 1; i < lista.size(); i++) {
        Vulnerabilidad actual = lista.get(i);
        int j = i - 1;
        while (j >= 0 && puntajeParaOrden(lista.get(j)) < puntajeParaOrden(actual)) {
            lista.set(j + 1, lista.get(j));
            j--;
        }
        lista.set(j + 1, actual);
    }
    return lista;
}
```

Fragmento 3. Ordenamiento por inserción de mayor a menor puntaje.

La búsqueda binaria aprovecha que la lista de hallazgos siempre está ordenada por número, porque cada hallazgo nuevo recibe el número siguiente y se agrega al final. En cada vuelta descarta la mitad de la lista, así que con 1.000 hallazgos necesita unas 10 comparaciones en lugar de 1.000. El resultado pasa después por buscarVisible(), que controla el acceso (si un cliente pide un número ajeno recibe el mismo mensaje que para un número inexistente, como pide CU13).

```
public Vulnerabilidad buscarPorNumero(int numero) {
    int izquierda = 0;
    int derecha = vulnerabilidades.size() - 1;
    while (izquierda <= derecha) {
        int medio = (izquierda + derecha) / 2;
        int actual = vulnerabilidades.get(medio).getIdVulnerabilidad();
        if (actual == numero) {
            return vulnerabilidades.get(medio);
        } else if (actual < numero) {
            izquierda = medio + 1;
        } else {
            derecha = medio - 1;
        }
    }
    return null;
}
```

Fragmento 4. Búsqueda binaria por número.

15.5 Extractos de ejecución

Los extractos son copia textual de los archivos de la carpeta salidas (donde falta un tramo se marca con [...]). En la sesión del administrador se registra el hallazgo 7 con sus ocho métricas y el sistema calcula 9,8 (crítica). Es el mismo vector del ejemplo de Shellshock que se usó en CP02, así que el valor coincide con el que publica FIRST.

```
Número de activo > 6
Título > Administración del firewall expuesta a Internet
Descripción > La interfaz de administración responde desde fuera de la red interna.
Tipo > Configuración insegura
Elemento afectado > Puerto 8443/TCP
Fecha de detección (dd/mm/aaaa) > 24/09/2026
¿Evaluar con CVSS 3.1 ahora? (S/N) > S
  AV Vector de ataque (N A L P) > N
  AC Complejidad del ataque (L H) > L
  PR Privilegios requeridos (N L H) > N
  UI Interacción del usuario (N R) > N
  S Alcance (U C) > U
  C Confidencialidad (H L N) > H
  I Integridad (H L N) > H
  A Disponibilidad (H L N) > H
Vulnerabilidad 7 registrada en estado PENDIENTE con CVSS 9,8 (Crítica).
```

Extracto 1. Registro de una vulnerabilidad con cálculo CVSS (salida_01_administrador.txt).

Después, al ordenar por puntaje, el hallazgo nuevo queda primero y el que no tiene evaluación queda último.

```
Opción > 3
Listado ordenado por puntaje CVSS, de mayor a menor
Proyectos visibles Portal web institucional, Gestión de expedientes, Infraestructura de oficina
Nro  Título                                          Estado           CVSS  Severidad
7    Administración del firewall expuesta a Internet  PENDIENTE         9,8  Crítica
4    Panel del NAS con credenciales por defecto       PENDIENTE         8,8  Alta
2    Inyección SQL en la búsqueda de expedientes      EN_TRATAMIENTO    8,1  Alta
3    XSS reflejado en el formulario de contacto       PENDIENTE         6,1  Media
1    TLS 1.0 y 1.1 habilitados                        CERRADA           5,9  Media
5    Divulgación de versión en encabezados HTTP       PENDIENTE         5,3  Media
6    Conexión a la base sin cifrado en la red interna PENDIENTE           -  Sin evaluar
7 vulnerabilidades en el listado.
```

Extracto 2. Listado ordenado por puntaje (salida_01_administrador.txt).

En la sesión del cliente ref.expedientes el listado muestra solo los hallazgos 2, 4 y 6, que son los de sus dos proyectos (el mismo resultado que la consulta C2 del script 03). El hallazgo 1 existe pero es de otro proyecto y el sistema responde igual que si no existiera (si contestara «sin permiso», ya le estaría confirmando que ese número existe). El menú del cliente tiene tres opciones, así que la 4 directamente no existe para él.

```
Usuario (SALIR para terminar) > ref.expedientes
Contraseña > ****************
Sesión iniciada. Referente de gestión de expedientes (CLIENTE)
[...]
Opción > 1
Listado por número
Proyectos visibles Gestión de expedientes, Infraestructura de oficina
Nro  Título                                          Estado           CVSS  Severidad
2    Inyección SQL en la búsqueda de expedientes      EN_TRATAMIENTO    8,1  Alta
4    Panel del NAS con credenciales por defecto       PENDIENTE         8,8  Alta
6    Conexión a la base sin cifrado en la red interna PENDIENTE           -  Sin evaluar
3 vulnerabilidades en el listado.
[...]
Opción > 2
Número de vulnerabilidad > 1
OPERACIÓN RECHAZADA. No hay una vulnerabilidad 1 entre sus proyectos.
[...]
Opción > 4
Opción inexistente, elija un número entre 0 y 3.
```

Extracto 3. Sesión del cliente con un número ajeno y una opción de gestión (salida_02_cliente.txt).

La tercera sesión prueba los errores de carga y las reglas del negocio. Ninguno corta el programa, cada error se informa y el menú vuelve a aparecer. La fecha se lee en modo estricto porque, con el modo por defecto, Java convierte sin avisar el 31/02/2026 en 28/02/2026 y el responsable terminaría con un plazo distinto del que se cargó sin que nadie lo note.

```
Usuario (SALIR para terminar) > dsrodriguez
Contraseña > ****************
ACCESO DENEGADO. Usuario o contraseña incorrectos.
[...]
Opción > abc
DATO INVÁLIDO. Escriba el número de la opción.
Opción > 9
Opción inexistente, elija un número entre 0 y 6.
Opción > 2
Número de vulnerabilidad > dos
DATO INVÁLIDO. Se esperaba un número.
[...]
Opción > 6
Número de vulnerabilidad > 4
Estado actual PENDIENTE
Estado nuevo (PENDIENTE, EN_TRATAMIENTO o CERRADA) > EN_TRATAMIENTO
Comentario > Se intenta iniciar el tratamiento sin asignar.
OPERACIÓN RECHAZADA. Falta responsable o fecha objetivo (primero hay que asignarlo).
[...]
Opción > 6
Número de vulnerabilidad > 5
Estado actual PENDIENTE
Estado nuevo (PENDIENTE, EN_TRATAMIENTO o CERRADA) > CERRADA
Comentario > Se intenta cerrar sin pasar por el tratamiento.
```

```
OPERACIÓN RECHAZADA. Transición no permitida de PENDIENTE a CERRADA.
[...]
Opción > 6
Número de vulnerabilidad > 5
Estado actual PENDIENTE
Estado nuevo (PENDIENTE, EN_TRATAMIENTO o CERRADA) > TERMINADA
DATO INVÁLIDO. No existe el estado TERMINADA.
[...]
Fecha objetivo (dd/mm/aaaa) > 31/02/2026
DATO INVÁLIDO. La fecha va como dd/mm/aaaa y tiene que existir.
[...]
  AV Vector de ataque (N A L P) > X
  Valor no válido, las opciones son N A L P.
```

Extracto 4. Errores de carga y reglas rechazadas (salida_03_errores.txt).

15.6 Capturas de ejecución

Las capturas muestran la salida real de app.MenuConsola al ejecutar las entradas guionadas de administrador y cliente. El visor herramientas/ver_ejecucion.py lanza Java y muestra su salida sin modificarla. Se utiliza para documentar la ejecución y no forma parte de la interfaz del sistema.

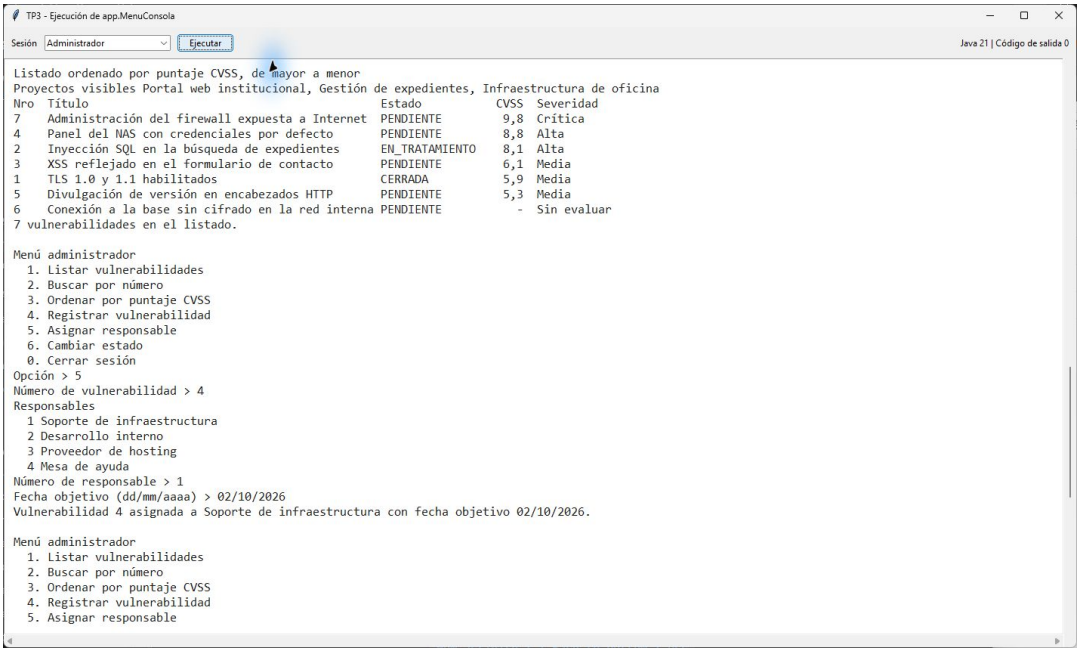


Figura 17. Ejecución del administrador. Se muestran los siete hallazgos ordenados por CVSS y la asignación del hallazgo 4. Captura propia.



Figura 18. Ejecución del cliente. Solo aparecen los hallazgos 2, 4 y 6, y se rechaza la consulta del hallazgo 1, que pertenece a otro proyecto. Captura propia.

15.7 Repositorio

El código fuente y las salidas completas, junto con las entradas que las generan y un README con los pasos para repetirlas, están en la carpeta TP3 del repositorio (https://github.com/rodriguezd0/semprac-seguimiento-vulnerabilidades/tree/main/TP3).

16. Versión final del sistema

En esta última entrega el prototipo pasa a ser la versión final del proyecto integrador. La lógica del TP3 se conserva, pero los datos ya no viven en listas en memoria sino en la base vulnerabilidades_db, a la que la aplicación accede con JDBC, y el menú de consola se reemplaza por las ventanas Swing de los prototipos del TP2. Cubre el inicio de sesión (CU11), la consulta con filtros según el rol (CU05 con CU13), el registro con cálculo CVSS (CU03 con CU12), el cambio de estado con la asignación (CU06 con CU04), el historial (CU07) y la exportación del listado (CU08), y suma la carga de hallazgos desde la planilla del área. Las altas de cuentas, proyectos, activos y responsables (CU01, CU02, CU09 y CU10) quedan fuera de esta versión y esos datos se cargan con el script 02, como corresponde a un prototipo que incluye solo algunas características del sistema final (Kendall y Kendall, 2011).

16.1 Patrón de diseño, selección y justificación

Elegí el patrón modelo-vista-controlador (MVC), que ya figuraba en la propuesta técnica (sección 5.2) y en el diagrama de despliegue. La lectura 4 lo justifica porque la interfaz es la parte que más cambia (los usuarios piden pantallas más simples, otros datos o funciones nuevas) y esos cambios no deberían tocar el núcleo funcional (Debrauwer, 2013, como se citó en *Patrones de diseño y JDBC*, s. f.). En este proyecto pasa algo parecido con la base, que en la operación real va a estar en otro equipo y quizás con otro usuario. Con MVC cada uno de esos cambios queda dentro de una sola capa.

Capa	Paquete y clases	Responsabilidad en el proyecto
Vista	vista (VentanaLogin, VentanaPrincipal y DialogoRegistro)	Muestra los datos y toma lo que carga el usuario. No tiene SQL ni reglas del negocio.
Controlador	controlador (ControladorVulnerabilidades)	Recibe cada pedido de las ventanas, controla la sesión y el rol, pide al modelo que valide y a los DAO que lean o guarden.

Capa	Paquete y clases	Responsabilidad en el proyecto
Modelo	modelo (11 clases)	Entidades y reglas del negocio (estados, fechas, asignación y cálculo CVSS).
Modelo, acceso a datos	dao (6 clases)	Conexión con MySQL y todas las sentencias SQL, con JDBC.
Apoyo	util (ArchivoTexto) y app (Aplicacion)	Archivos de texto y arranque (crea los DAO, el controlador y la primera ventana).

La lectura 4 ubica el acceso a la base dentro del modelo (*Patrones de diseño y JDBC*, s. f.). Lo separé en el paquete dao para que las entidades no conozcan SQL, lo que también responde a la devolución del TP2 sobre no concentrar lógica de gestión en las entidades (Vulnerabilidad solo valida sus propias reglas, el controlador coordina y el DAO guarda). La figura 19 muestra las clases de cada capa, con las dependencias siempre hacia abajo (ninguna ventana habla directamente con un DAO).

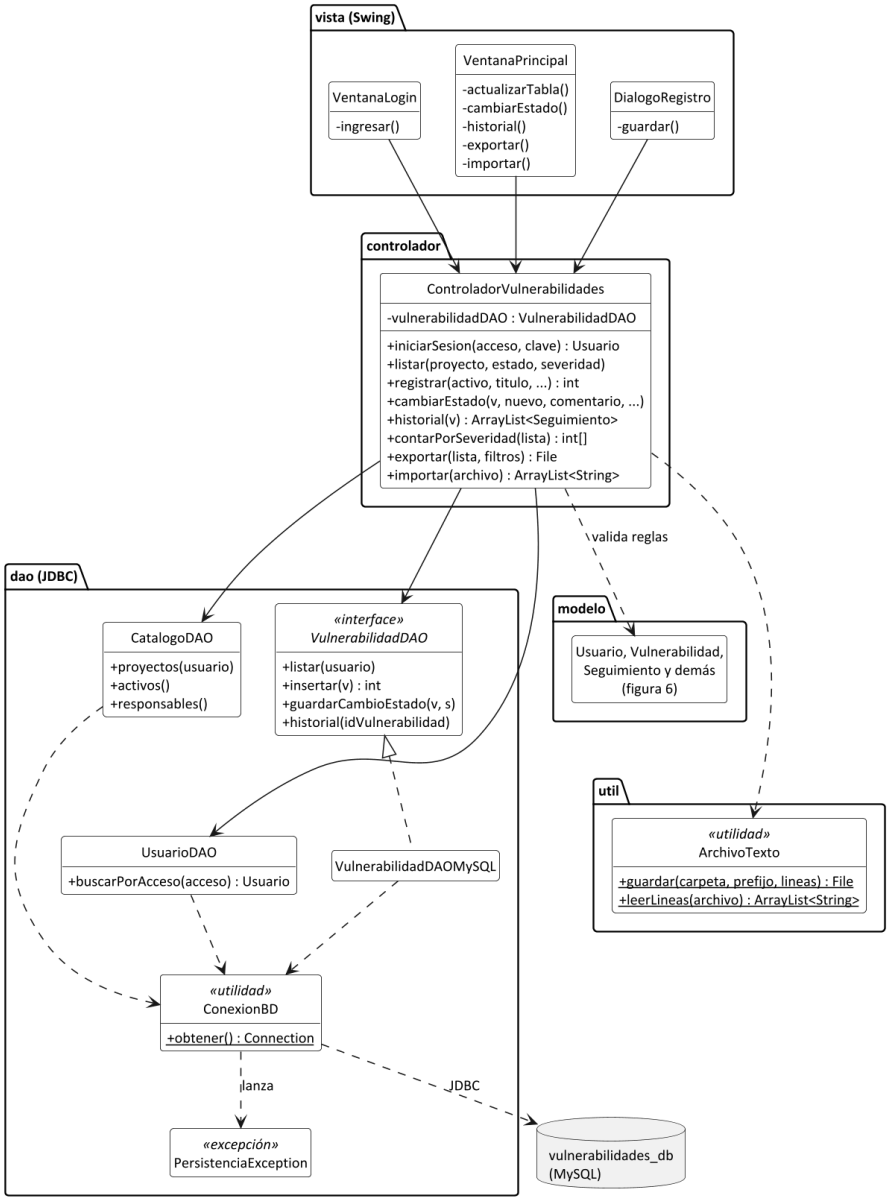


Figura 19. Clases de la versión final agrupadas por capa del patrón MVC. Elaboración propia.

El alta de un hallazgo sirve de ejemplo del recorrido completo. DialogoRegistro arma el vector CVSS con los combos y llama a registrar() del controlador. El controlador revisa el rol y la fecha, crea la Vulnerabilidad (su constructor valida título, tipo y largos) y le pide a VulnerabilidadDAOMySQL que la

inserte. La base devuelve el número asignado y la ventana lo informa.

16.2 Persistencia y consultas con JDBC

Siguiendo la lectura 4, ConexionBD carga el driver con Class.forName() y pide la conexión a DriverManager.getConnection() (Grousard, 2010, como se citó en *Patrones de diseño y JDBC*, s. f.). Cada operación abre su conexión dentro de un try con recursos, así se cierra sola aunque haya un error. Todas las sentencias son PreparedStatement con parámetros y nunca texto concatenado (justamente el hallazgo 2 de los datos de prueba es una inyección SQL). La URL, el usuario y la clave se pueden cambiar al ejecutar, sin recompilar.

Operación	Clase y método	Sentencia
Establecer la conexión	ConexionBD, obtener()	DriverManager.getConnection(url, usuario, clave)
Iniciar sesión (CU11)	UsuarioDAO, buscarPorAcceso()	SELECT de usuario por nombre_acceso
Consultar (CU05 y CU13)	VulnerabilidadDAOMySQL, listar()	SELECT con JOIN a activo y proyecto, más WHERE p.id_cliente = ? si la cuenta es cliente
Registrar (CU03)	VulnerabilidadDAOMySQL, insertar()	INSERT con RETURN_GENERATED_KEYS para recuperar el número
Actualizar (CU06 y CU04)	VulnerabilidadDAOMySQL, guardarCambioEstado()	UPDATE de vulnerabilidad e INSERT de seguimiento en una transacción
Historial (CU07)	VulnerabilidadDAOMySQL, historial()	SELECT de seguimiento con JOIN a usuario
Catálogos	CatalogoDAO	SELECT de proyectos, activos y responsables

El cambio de estado es la operación más delicada, porque el UPDATE del hallazgo y el INSERT del seguimiento tienen que quedar los dos o ninguno (RNF03). El DAO desactiva el autocommit, ejecuta las dos sentencias y recién ahí confirma con commit(). Si cualquiera falla hace rollback() y la base queda como estaba. Sin la transacción, un error entre las dos sentencias dejaría un hallazgo cerrado sin saber quién lo cerró ni por qué, que es justo el problema de la planilla que el sistema viene a resolver.

```
try (Connection con = ConexionBD.obtener()) {
    con.setAutoCommit(false);
    try (PreparedStatement actualizar = con.prepareStatement(SQL_ESTADO);
         PreparedStatement registrar = con.prepareStatement(SQL_SEGUIMIENTO)) {
        ...
        actualizar.executeUpdate();
        ...
        registrar.executeUpdate();
        con.commit();
    } catch (SQLException e) {
        con.rollback();
        throw e;
    }
} catch (SQLException e) {
    throw new PersistenciaException("No se pudo guardar el cambio de estado, la base quedó como estaba.", e);
}
```

Fragmento 5. Transacción del cambio de estado en VulnerabilidadDAOMySQL.

Para presentar los resultados, actualizarTabla() de VentanaPrincipal recibe del controlador el ArrayList ya filtrado y ordenado y lo carga en la JTable, una fila por hallazgo. Después de cada alta, cambio de estado o importación la ventana vuelve a leer la base, así lo que se ve es siempre lo guardado (figura 20).

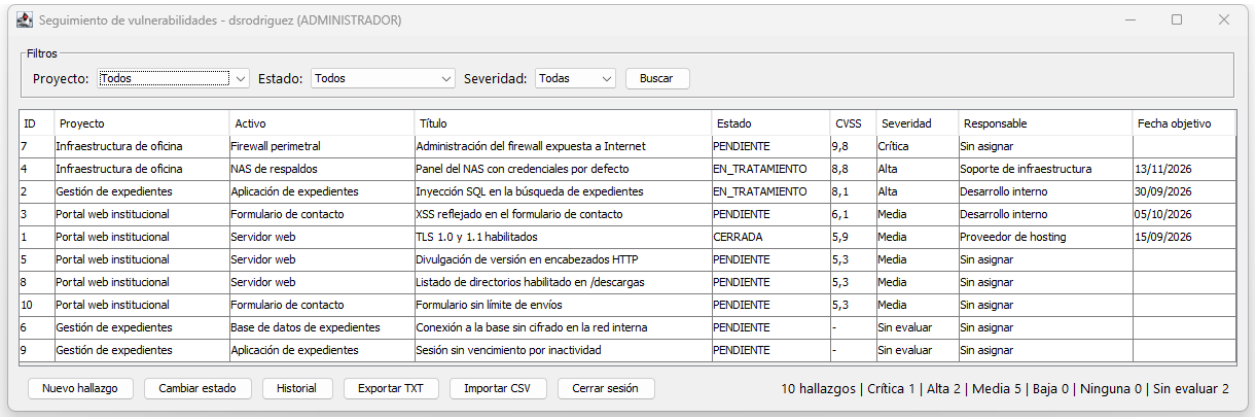


Figura 20. Ventana principal del administrador con los diez hallazgos leídos de MySQL y ordenados por puntaje. Captura propia.

16.3 Excepciones en la interacción con la base

La lectura 4 distingue las excepciones verificadas, que el compilador obliga a tratar, de las no verificadas (*Patrones de diseño y JDBC*, s. f.). Las uso con ese criterio. PersistenciaException extiende Exception, así cada método que toca la base la declara con throws y quien lo llama no se puede olvidar de atraparla. Guarda la SQLException original como causa y trae un mensaje que se le puede mostrar al usuario sin detalles técnicos. Las reglas del negocio siguen con ReglaNegocioException del TP3, que es no verificada porque representa un dato mal cargado y no una falla del sistema.

Situación	Qué se lanza y dónde	Dónde se atrapa y qué ve el usuario
Falta el driver de MySQL	ClassNotFoundException en ConexionBD, traducida a PersistenciaException	Ventana de acceso, con el motivo en rojo
MySQL apagado o sin la base	SQLException en getConnection(), traducida a PersistenciaException	Ventana de acceso (figura 22, CP10)
Falla un UPDATE o un INSERT	SQLException en el DAO, rollback() y PersistenciaException	Cuadro «Error de la base de datos». El hallazgo conserva su estado (CP08)
Regla del negocio no cumplida	ReglaNegocioException en Vulnerabilidad o en el controlador	Aviso «Operación rechazada» con el motivo
Fecha imposible (31/02)	DateTimeParseException al leer el campo	Aviso con el formato esperado
Problema con un archivo	IOException en ArchivoTexto	Cuadro de error con el nombre del archivo

La figura 21 muestra el CU06 con sus dos caminos de error. En VentanaLogin se usa además finally para borrar la contraseña del campo, tanto si el ingreso sale bien como si falla (figura 22).

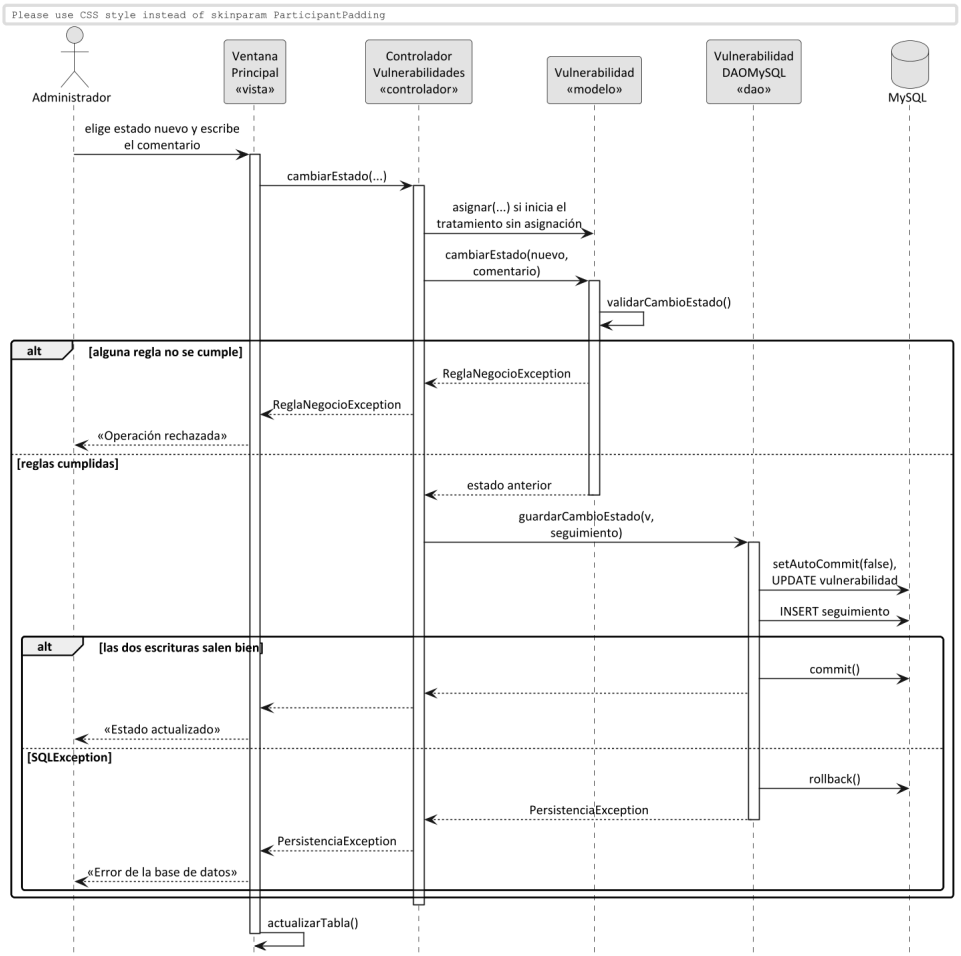


Figura 21. Diagrama de secuencia de diseño de CU06 con la transacción y las excepciones. Elaboración propia.

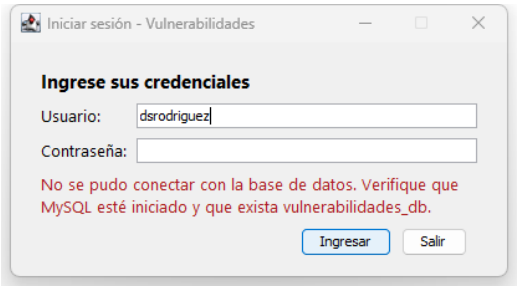


Figura 22. Inicio de sesión con el servidor de base apagado. Captura propia.

16.4 Clases abstractas e interfaces

Usuario sigue siendo abstracta, como en el TP3, y ahora la ventana principal arma sus botones recorriendo usuario.opcionesMenu(). El administrador recibe cinco botones y el cliente uno solo (Historial), sin que la ventana pregunte de qué clase es la cuenta (figuras 20 y 23). La interfaz VulnerabilidadDAO declara las cuatro operaciones sobre los hallazgos y VulnerabilidadDAOMySQL las implementa. El controlador guarda el atributo con el tipo de la interfaz, por eso podría recibir otra implementación (por ejemplo, una con las listas en memoria del TP3 para probar sin servidor) sin cambiar el controlador ni las ventanas. Es la misma idea de JDBC, que según la lectura 4 es sobre todo un conjunto de interfaces (Grousard, 2010, como se citó en *Patrones de diseño y JDBC*, s. f.). Connection, PreparedStatement y ResultSet son interfaces y cada fabricante las implementa en su driver.

```
public interface VulnerabilidadDAO {
    ArrayList<Vulnerabilidad> listar(Usuario usuario) throws PersistenciaException;
    int insertar(Vulnerabilidad vulnerabilidad) throws PersistenciaException;
    void guardarCambioEstado(Vulnerabilidad vulnerabilidad, Seguimiento seguimiento) throws PersistenciaException;
    ArrayList<Seguimiento> historial(int idVulnerabilidad) throws PersistenciaException;
}

public class VulnerabilidadDAOMySQL implements VulnerabilidadDAO {
    ...
}

// En ControladorVulnerabilidades
private final VulnerabilidadDAO vulnerabilidadDAO;
```

Fragmento 6. Interfaz VulnerabilidadDAO, su implementación y el atributo del controlador.

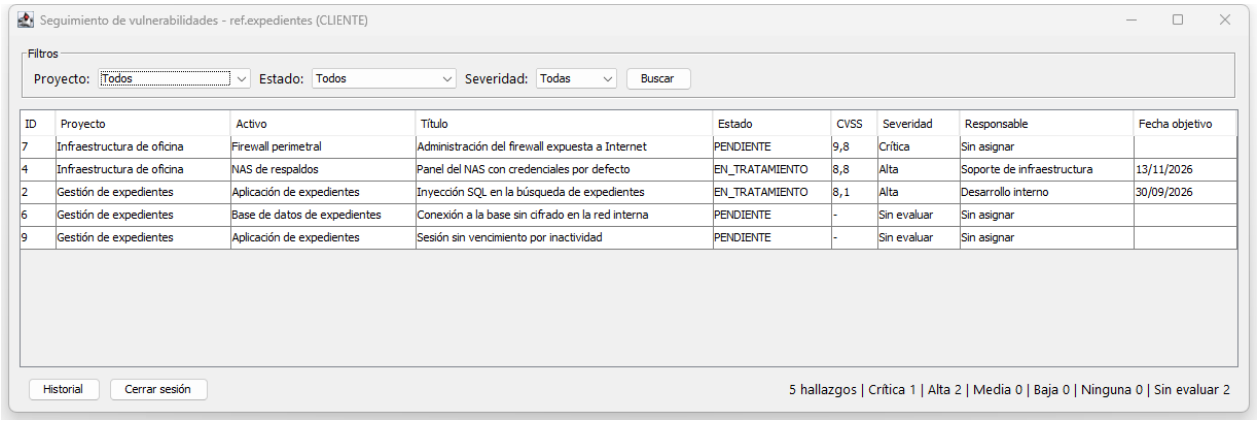


Figura 23. Ventana del cliente ref.expedientes. Solo ve los hallazgos de sus dos proyectos y tiene un único botón de consulta. Captura propia.

16.5 Arreglos y ArrayList

Los uso de forma complementaria según si la cantidad se conoce de antemano. Lo que viene de la base o de un archivo va en un ArrayList, porque la cantidad recién se sabe al leer, y lo que tiene un tamaño fijo por definición va en un arreglo.

Estructura	Dónde	Por qué
ArrayList<Vulnerabilidad> y ArrayList<Seguimiento>	Resultados de los DAO y filtros del controlador	La cantidad de filas depende de la consulta.
ArrayList<String>	Líneas de los archivos e informe de la importación	No se sabe cuántas líneas tiene el archivo.
String[] SEVERIDADES e int[] conteo	contarPorSeveridad() y el resumen al pie de la ventana	Las categorías de severidad son siempre seis.
String[] METRICAS y String[][] OPCIONES	Panel CVSS de DialogoRegistro	Las métricas base son siempre ocho.
Object[] fila y Object[][] datos	Filas de la tabla principal y del historial	JTable recibe arreglos.
String[] campos	Cada fila del CSV partida con split()	La planilla tiene ocho columnas fijas.
String[] opcionesMenu()	Botones de cada rol	Cada rol tiene una cantidad fija de opciones.

Para filtrar recorro el ArrayList con un Iterator, como propone la lectura 4 (*Patrones de diseño y JDBC*, s. f.). Quitar elementos de la lista dentro de un for-each lanza ConcurrentModificationException, en cambio remove() del iterador los quita sin cortar el recorrido.

```
// En ControladorVulnerabilidades.contarPorSeveridad()
int[] conteo = new int[SEVERIDADES.length];
for (Vulnerabilidad v : lista) {
    for (int i = 0; i < SEVERIDADES.length; i++) {
        if (SEVERIDADES[i].equals(v.getSeveridad())) {
            conteo[i]++;
        }
    }
}

// En ControladorVulnerabilidades.listar()
Iterator<Vulnerabilidad> it = lista.iterator();
while (it.hasNext()) {
    Vulnerabilidad v = it.next();
    ...
    if (!(pasaProyecto && pasaEstado && pasaSeveridad)) {
        it.remove();
    }
}
```

Fragmento 7. Arreglo de conteo por severidad y filtro del ArrayList con Iterator.

16.6 Manejo de archivos

La consigna deja los archivos como optativos, pero los usé en dos funciones. La primera es la exportación del listado (CU08), que guarda lo que se ve en pantalla en un archivo de texto nuevo dentro de la carpeta exportaciones y después lo vuelve a leer del disco para mostrarlo (figura 24). La segunda es la carga inicial desde la planilla del área guardada como CSV, con el botón «Importar CSV». Así los hallazgos que hoy están en la planilla pasan al sistema sin cargarlos de nuevo a mano, que es parte de la transición del PUD.

Las dos funciones usan la clase ArchivoTexto, que aplica la diferencia que marca la lectura 4 entre la clase File y los flujos (Sánchez Asenjo, 2010, como se citó en *Patrones de diseño y JDBC*, s. f.). File solo crea la carpeta y revisa si el archivo ya existe. Para escribir se encadenan FileOutputStream (bytes al disco), OutputStreamWriter (texto en UTF-8) y BufferedWriter (junta las escrituras chicas), y para leer se recorre el camino inverso con BufferedReader, que entrega una línea completa por vez.

```
// En ArchivoTexto.guardar()
if (!carpeta.exists() && !carpeta.mkdirs()) {
    throw new IOException("No se pudo crear la carpeta " + carpeta.getAbsolutePath());
}
...
try (BufferedWriter salida = new BufferedWriter(
        new OutputStreamWriter(new FileOutputStream(archivo), StandardCharsets.UTF_8))) {
    for (String linea : lineas) {
        salida.write(linea);
        salida.newLine();
    }
}

// En ControladorVulnerabilidades.importar()
String[] campos = lineas.get(n).split(";", -1);
```

Fragmento 8. Escritura con flujos encadenados y lectura de una fila del CSV.

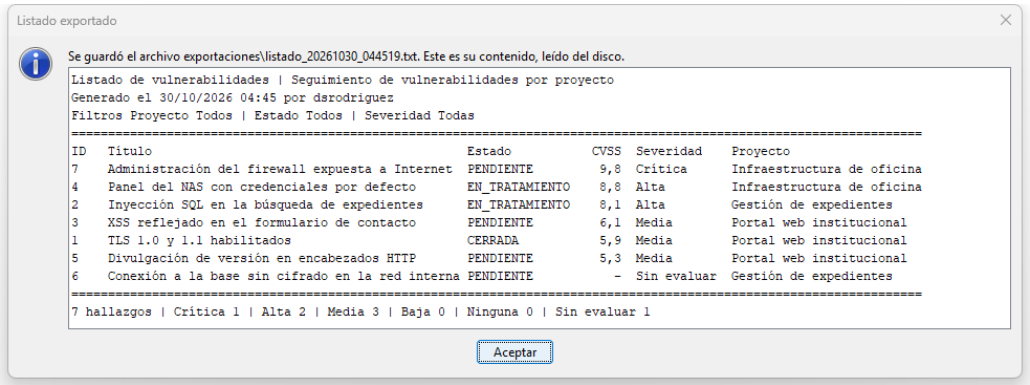


Figura 24. Listado exportado y leído de nuevo desde el disco (CU08). Captura propia.

En la importación cada fila se parte en un arreglo de ocho campos y pasa por el mismo registrar() del alta manual, así tiene las mismas validaciones. Una fila mal cargada se informa y no frena a las demás. Con la planilla de ejemplo del repositorio se registraron 3 de las 6 filas y se rechazaron las otras tres (una fecha imposible, un activo que no existe y un vector con un valor que no es de CVSS), como muestra la figura 25.

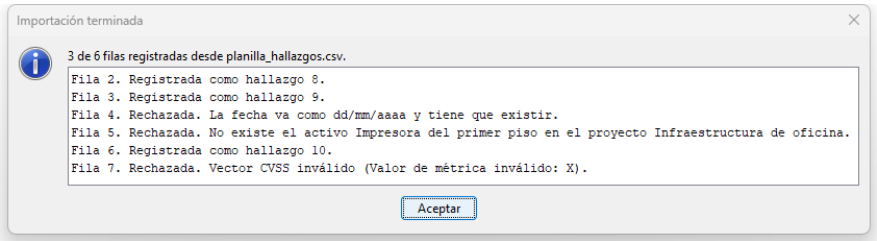


Figura 25. Informe de la importación de la planilla. Captura propia.

16.7 Ejecución y pruebas de la versión final

Se compila con el JDK 21 y MySQL Connector/J 8.4.0, que está en la carpeta lib del repositorio. Con todas las advertencias de javac activadas la compilación terminó sin errores ni advertencias y generó las 23 clases del sistema. La base es la del TP2 (scripts 01 y 02 sin cambios) y las pruebas se hicieron con el MySQL de XAMPP (MariaDB 10.4). Para dejar evidencia armé el programa auxiliar PruebasTP4, que abre las ventanas reales, escribe en los campos y pulsa los botones como un usuario, saca cada captura y después consulta la base para comprobar lo que quedó guardado. No forma parte del sistema, es una herramienta de prueba.

Código	Tipo	Qué verifica	Esperado	Obtenido
CP05	Sistema	Hallazgos y botones del cliente ref.expedientes	Hallazgos 7, 4, 2, 6 y 9, sin botones de gestión	Igual al esperado
CP07	Integración	Alta desde la ventana con las ocho métricas	Hallazgo 7 en la base, PENDIENTE y con 9,8	Igual al esperado
CP08	Integración	Cambio de estado del hallazgo 4 y rollback forzado con un usuario inexistente	EN_TRATAMIENTO con un seguimiento más. Con el error, la base sin cambios	Igual al esperado (error 1452 y rollback)
CP09	Integración	Exportación a TXT e importación del CSV	Archivo de 14 líneas. 3 filas registradas y 3 rechazadas	Igual al esperado
CP10	Sistema	Inicio de sesión con el servidor apagado	Mensaje de error y contraseña borrada	Igual al esperado

Antes de cerrar la entrega se revisó el código buscando casos de borde y aparecieron siete defectos menores (por ejemplo, un vector CVSS de más de 44 caracteres que la base habría guardado recortado, o una planilla guardada con otra codificación que cambiaba las tildes). Se corrigieron con el ciclo de la sección 11.5 y después se repitieron todas las pruebas.

Las 17 verificaciones dieron el resultado esperado y sus salidas completas están en la carpeta salidas del repositorio. Al terminar, la consulta 3 del script 03_verificacion.sql confirma que el estado de cada hallazgo coincide con el último seguimiento, que es la postcondición de CU06. La figura 26 muestra el alta del hallazgo 7 con su panel CVSS, que es el recorrido del video.

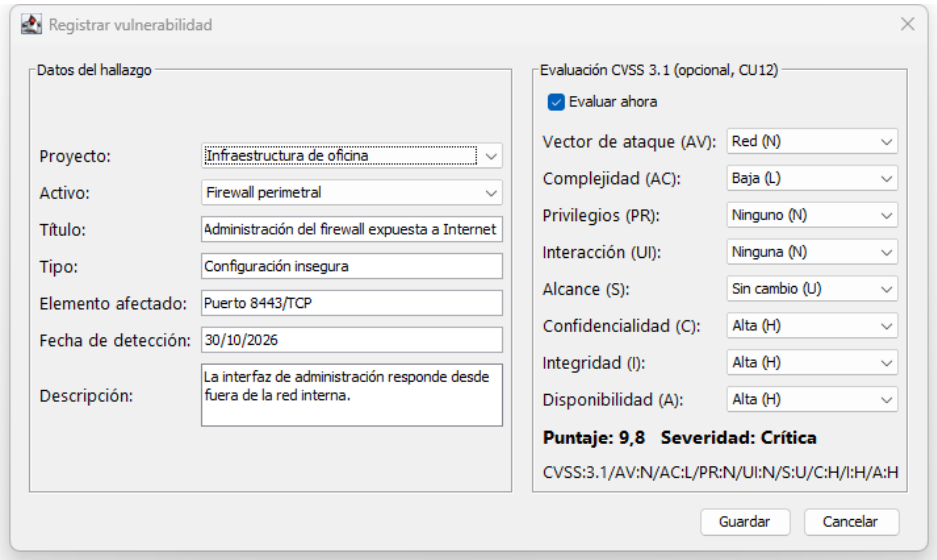


Figura 26. Registro de un hallazgo con el cálculo CVSS 3.1 (CU03 extendido por CU12). Captura propia.

16.8 Correcciones respecto de las entregas anteriores

Las observaciones de la devolución del TP1 se corrigieron en el TP2 (secciones 1 a 7). La tabla resume las del TP2, las pautas del encuentro del 20 de octubre y el resultado de la devolución del TP3.

Observación	Corrección
Las entidades no deben concentrar lógica de gestión (TP2)	Las entidades solo validan sus reglas. La coordinación está en el controlador y la persistencia en los DAO (16.1).
Completar las definiciones de la implementación (TP2)	La sección 10 describe los seis paquetes, el driver y el archivo .jar de la versión final.
Detallar los resultados esperados con valores concretos (TP2)	Las pruebas CP05 y CP07 a CP10 indican el valor esperado y el obtenido (16.7).
Mantener la coherencia con el script (TP2)	Los DAO usan las mismas tablas y columnas de los scripts 01 y 02, que no cambiaron.
Dejar algunas consultas de referencia en el documento y el resto en archivos aparte (TP2)	La sección 13.6 muestra la consulta principal con su resultado (figura 15) y la 13.8 indica dónde están los scripts completos.
Agregar el enlace de datos en las comunicaciones (TP2)	La tabla de la sección 14 suma la capa de enlace y física (Ethernet IEEE 802.3 y Wi-Fi IEEE 802.11).
Diagrama de clases de diseño coherente con lo implementado (encuentro del 20/10)	La figura 6 se actualizó con los métodos y la excepción del código final, y la figura 19 agrega las demás capas.
Nota 100/100 del 25/10/2026, sin observaciones pendientes (TP3)	El profesor Pablo Alejandro Virgolini otorgó 20/20, 45/45, 20/20 y 15/15. Destacó la explicación del desarrollo en Java, ReglaNegocioException, MenuConsola y la estructura orientada a objetos. La versión final conserva esa base e integra Swing con MySQL (sección 16).

16.9 Repositorio y video

El código completo, los scripts SQL, la planilla de ejemplo, las salidas y las capturas están en la carpeta TP4 del repositorio público, junto con un README que explica cómo crear la base y ejecutar la aplicación (con el archivo .jar o compilando las fuentes en NetBeans o en una consola).

Repositorio https://github.com/rodriguezd0/semprac-seguimiento-vulnerabilidades/tree/main/TP4

Video de presentación
https://drive.google.com/file/d/1WOmHzJpT8UlJvwH-wRaepPd_MIHo2izk/view?usp=sharing

17. Trazabilidad y referencias

La tabla sigue cada requerimiento que cubre la versión final desde el caso de uso hasta la prueba, el prototipo de interfaz y los métodos que lo implementan.

RF / CU	Análisis	Diseño	Tablas	Prueba	Interfaz	Versión final
RF03 / CU03	Pantalla y gestor de registro	Vulnerabilidad	vulnerabilidad	CP04, CP07	Figuras 11 y 26	DialogoRegistro.guardar(), registrar(), insertar()
RF12 / CU12	Calculadora CVSS	CalculadoraCvss31	vulnerabilidad	CP01, CP02, CP07	Figuras 11 y 26	vector(), calcularPuntaje(), getSeveridad()
RF04 / CU04	Gestor de cambio de estado	Vulnerabilidad, asignar()	vulnerabilidad	CP04, CP08	No aplica	cambiarEstado() del controlador, asignar()
RF05 / CU05, CU13	Pantalla y gestor de consulta	Cliente, Proyecto	proyecto, activo, vulnerabilidad	CP05	Figuras 10 y 20	listar() del controlador y del DAO
RF06 / CU06	Pantalla y gestor de cambio de estado	Vulnerabilidad, validarCambioEstado()	vulnerabilidad, seguimiento	CP03, CP08	No aplica	cambiarEstado(), guardarCambioEstado()
RF07 / CU07	Gestor de consulta	Seguimiento	seguimiento	CP08	No aplica	historial()
RF08 / CU08	Pantalla de consulta	Listado filtrado	No aplica	CP09	Figura 24	exportar(), ArchivoTexto.guardar()
RF11 / CU11	Pantalla y gestor de sesión	Usuario, verificarClave()	usuario	CP06, CP10	Figuras 9 y 22	iniciarSesion(), buscarPorAcceso()

El repositorio completo del proyecto (carpetas TP1, TP2, TP3 y TP4) es https://github.com/rodriguezd0/semprac-seguimiento-vulnerabilidades.

Referencias

FIRST. (2019a). *Common Vulnerability Scoring System v3.1: Specification document*. https://www.first.org/cvss/v3.1/specification-document

FIRST. (2019b). *Common Vulnerability Scoring System v3.1: Examples*. https://www.first.org/cvss/v3.1/examples

Kendall, K. E., y Kendall, J. E. (2011). *Análisis y diseño de sistemas* (8.ª ed.). Pearson Educación.

Ley 25.326 de Protección de los Datos Personales. (2000). Boletín Oficial de la República Argentina.

Patrones de diseño y JDBC [Lectura 4 del Seminario de Práctica de Informática]. (s. f.). Universidad Siglo 21. Canvas, curso 46312.

Proceso unificado de desarrollo [Lectura 2 del Seminario de Práctica de Informática]. (s. f.). Universidad Siglo 21. Canvas, curso 46312.

Programación orientada a objetos [Lectura 3 del Seminario de Práctica de Informática]. (s. f.). Universidad Siglo 21. Canvas, curso 46312.

Sistemas de información [Lectura 1 del Seminario de Práctica de Informática]. (s. f.). Universidad Siglo 21. Canvas, curso 46312.

Anexo A. Preguntas de elicitación

Las respuestas sintetizan el relevamiento que hice en el área tecnológica durante septiembre de 2026, sin nombres ni datos personales de los participantes.

A.1 Entrevista al responsable del área tecnológica

Pregunta	Síntesis de la respuesta
1. ¿Cómo se detectan hoy los hallazgos de seguridad?	Por revisiones de configuración, alertas de los equipos y avisos de usuarios.
2. ¿Dónde se registran y qué datos se anotan?	En una planilla del área, con el recurso, la descripción y una gravedad estimada. Responsable y avances quedan en los correos.
3. ¿Quién decide la gravedad y con qué criterio?	Quien anota el hallazgo, según su experiencia. No hay un método común.
4. ¿Quién corrige y cómo se asigna la tarea?	Depende del recurso (soporte interno, desarrollo o el proveedor de hosting) y se avisa por correo o mensajería.
5. ¿Cuándo se considera cerrado un hallazgo?	Cuando se verifica la corrección sobre el activo, por ejemplo repitiendo la prueba que lo detectó.
6. ¿Qué pasa si un hallazgo cerrado vuelve a aparecer?	Se anota como una fila nueva y se pierde la relación con el caso anterior.
7. ¿Qué información no debería ver un referente?	Los hallazgos de proyectos que no son suyos.
8. ¿Qué método de severidad resultaría aceptable?	Uno estándar y reconocido. Se acordó usar CVSS 3.1 base.

A.2 Cuestionario a referentes de proyecto

Pregunta	Síntesis de las respuestas
1. ¿Con qué frecuencia necesita conocer el estado de los hallazgos de su proyecto?	Una vez por semana o cuando hay un cambio importante.
2. ¿Qué datos necesita ver?	Título, estado, severidad, responsable y fecha objetivo. El detalle técnico, solo a veces.
3. ¿Le sirve ver el historial de cambios?	Sí, para saber qué se hizo y por qué se reabrió algo.
4. ¿Necesita modificar datos?	No, solo consultar.